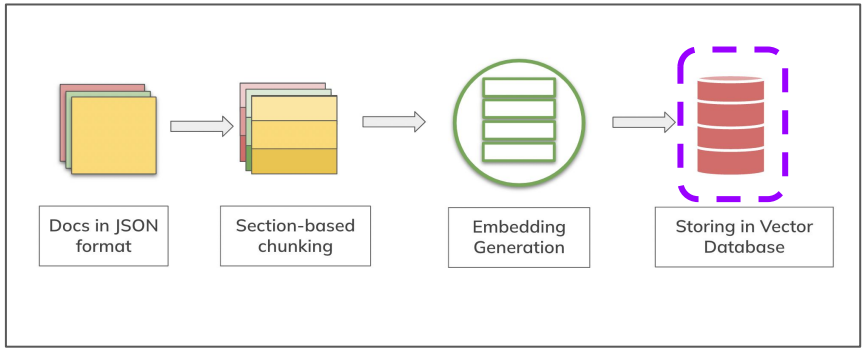
Retrieval Augmented Generation (RAG)
Module 2 Lecture 5

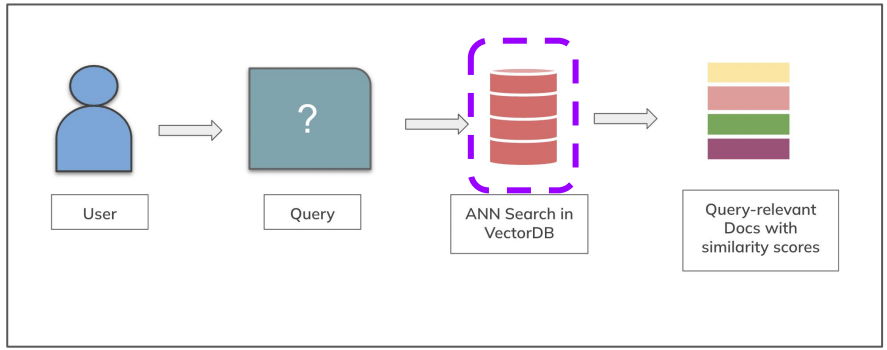
Pawan Goyal
Department of Computer Science and Engineering
IIT Kharagpur

RAG with Vector DB - *Recap From the last lecture*

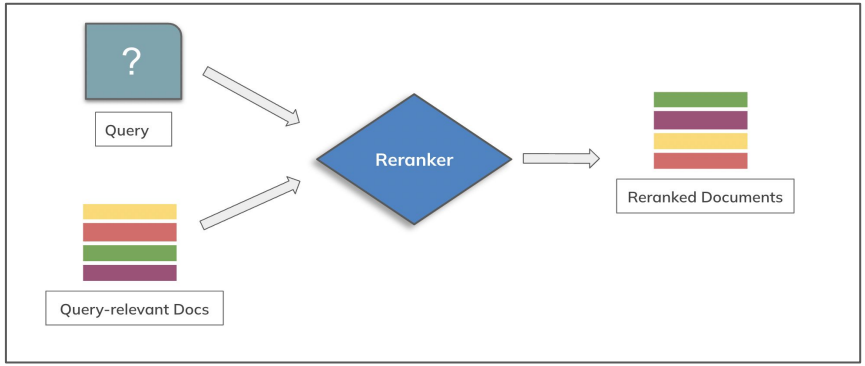
Pre-Retrieval



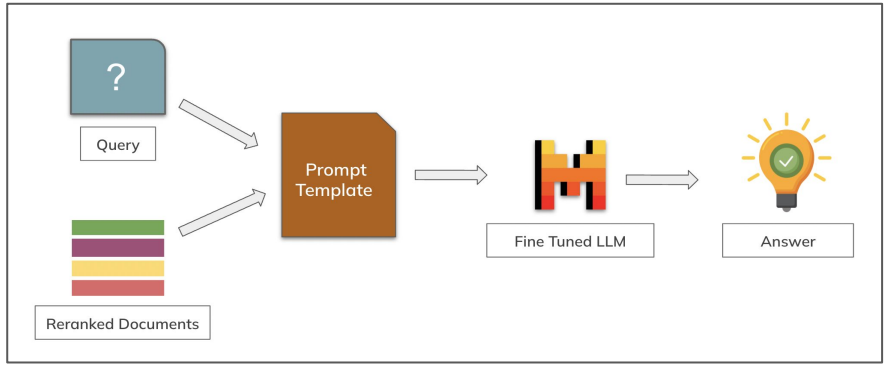
Retrieval



Post-Retrieval



Augmentation and Generation

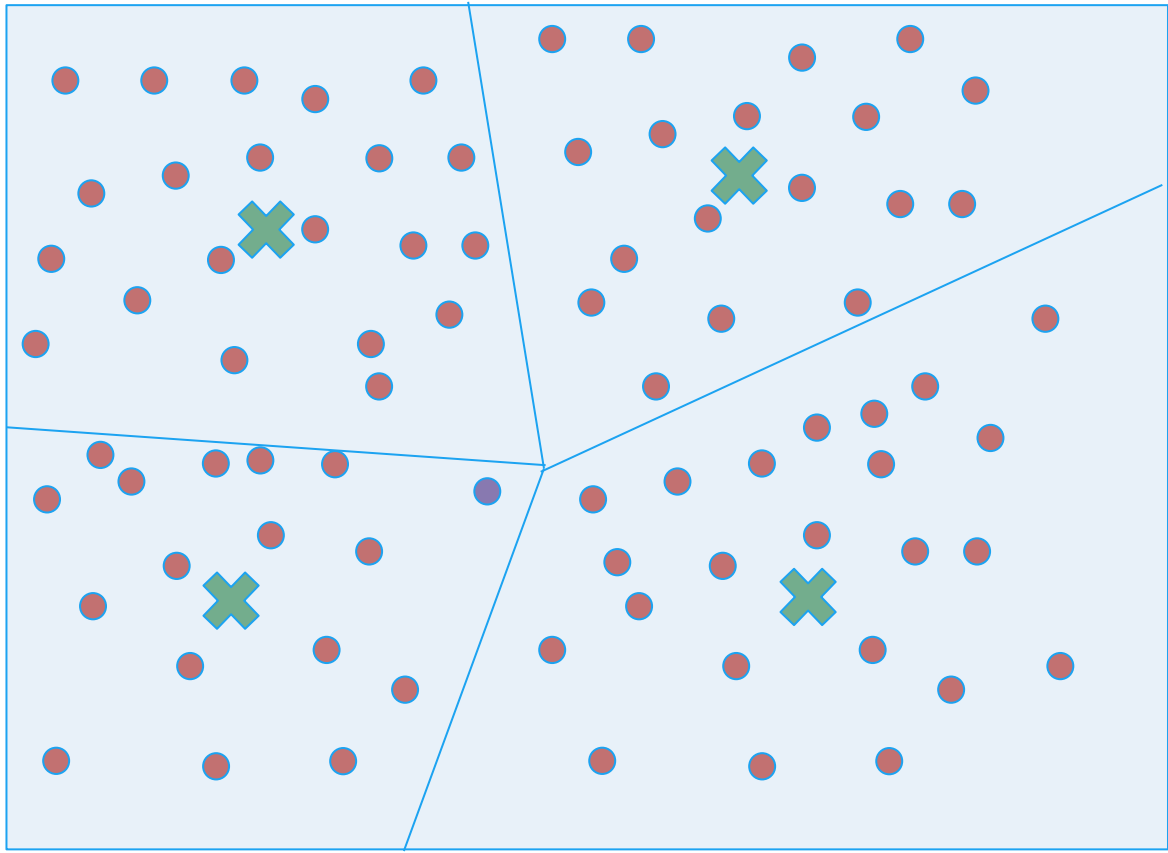


https://ai.meta.com/blog/retrieval-augmented-generation-streamlining-the-creation-of-intelligent-natural-language-processing-models/

IVF: Inverted file for vector Database - *Recap From the last lecture*

Retrieval (The Search)

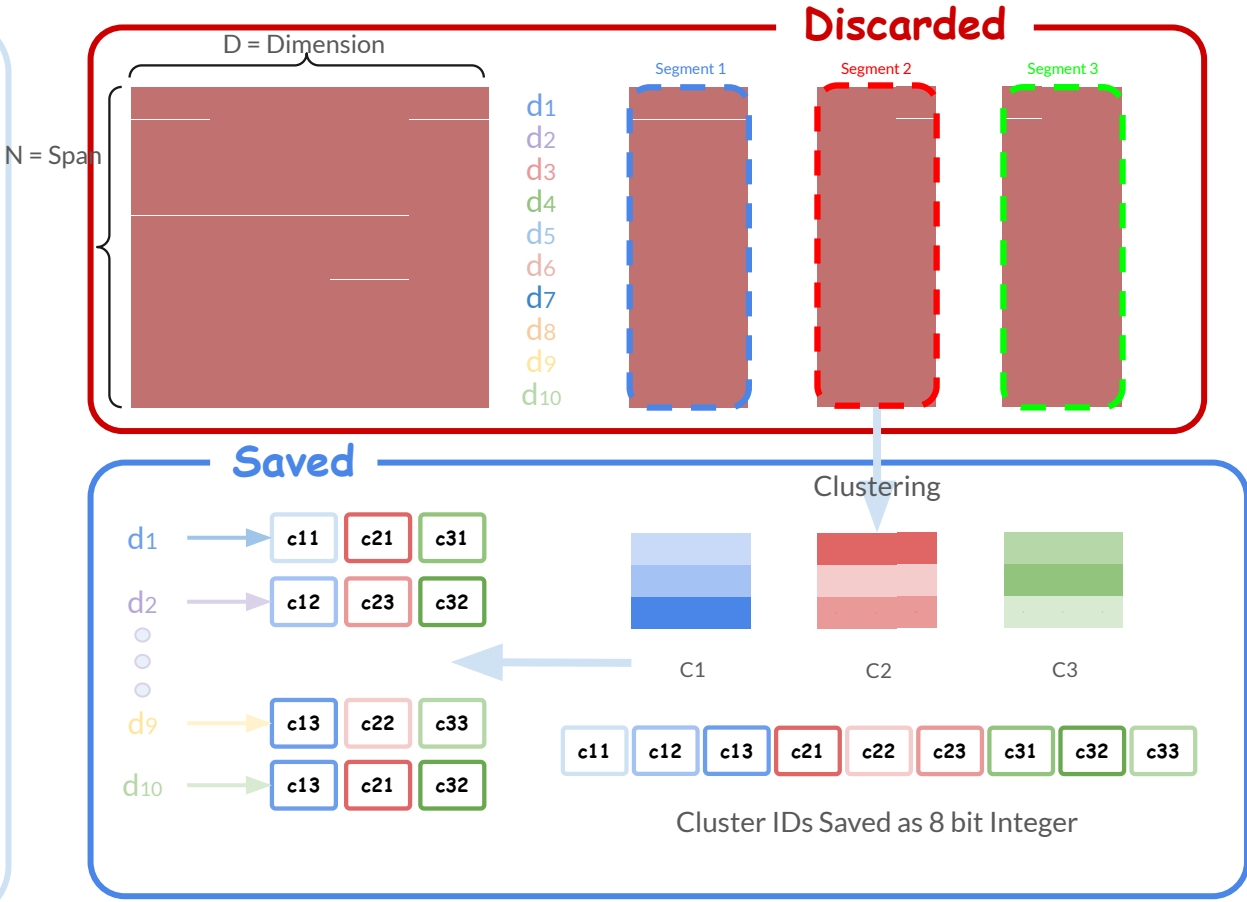
- **Coarse Search:** Compare the query **only** against the k centroids to find the closest match.
- **Fine Search:** Scan **only** the vectors inside that specific bucket (ignoring the rest)
- **nprobe** : #clusters (centroids) the algorithm evaluates during an ANN search (*=1 by default*)
- **Outcome:** Reduces search scope from O(N) to approximately O(N/k)



PQ: Product Quantisation (Indexing)- *Recap From the last lecture*

Indexing (Compression Phase)

- **Split (Sub-vectors):** Divide the high-dimensional original vector (e.g., 128D) into m smaller sub-vectors (e.g., 8 chunks of 16D).
- **Train (Codebooks):** Run K-Means independently on each sub-space to find K (eg. 256) representative centroids (the "Codebook").
- **Encode:** Replace each sub-vector with the **ID** (0-255) of its nearest centroid.
- **Result:** High-precision floats are converted into small byte codes, reducing memory usage by 95%+ (e.g., 4KB →64 bytes).



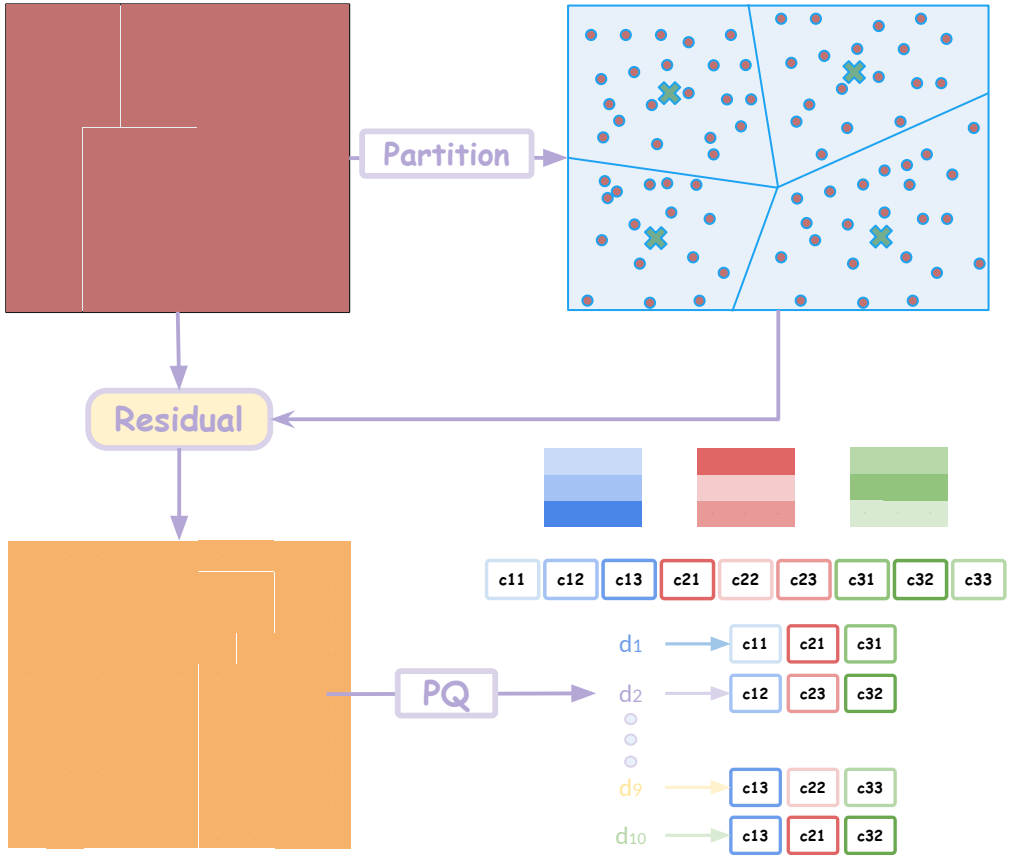
IVF + PQ (*Combining the two*)

IVF and PQ each solve two different problems:

1. **IVF** solves **"Too many vectors to check"** (by clustering).
2. **PQ** solves **"Vectors are too big to store"** (by compression).

IVF+PQ Indexing (Construction)

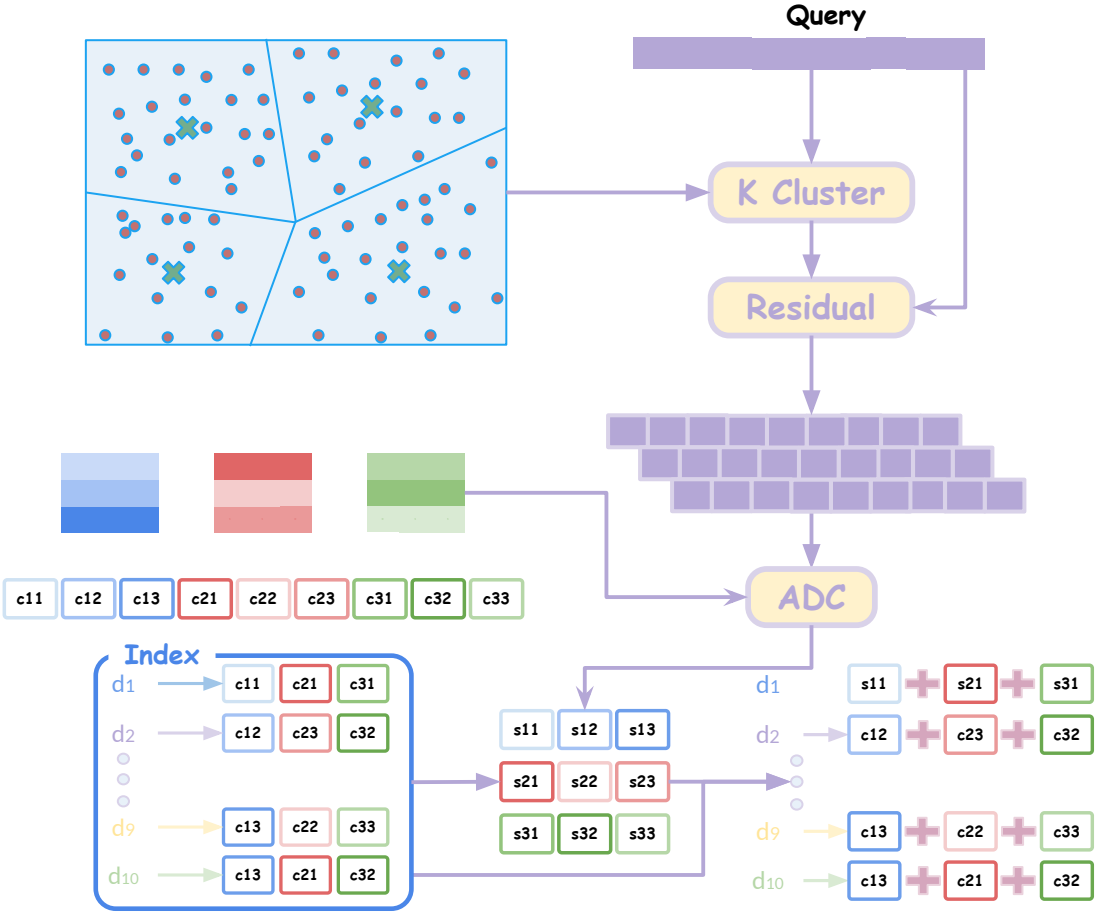
- 1. **Coarse (IVF):** Run K-Means (e.g., 1024 clusters) on the dataset to find coarse centroids.
- 2. Assign each vector to its nearest centroid, then calculate the difference vector (Residual = Vector - Centroid).
- 3. **Fine (PQ):** Split the **residual vector** into sub-vectors and train the PQ codebooks on these residuals.
- 4. Save the PQ codes (byte IDs) into the specific Inverted List for that centroid.



IVF + PQ

IVF+PQ Search (Retrieval)

- **1. Coarse (IVF):** Compare the query against the coarse centroids to find the nprobe closest buckets (e.g., top 10).
- **2.** For each selected bucket, shift the query vector relative to that bucket's centroid (Queryshifted=Query−Centroid).
- **3. Build Tables (ADC):** Construct the PQ distance lookup tables using these shifted queries.
- **4. Scan:** Iterate through the PQ codes in the selected buckets, summing values from the lookup table to approximate distance.



IVF + PQ: why the approximation works

IVF prunes **how many** vectors you score; PQ shrinks **how many bytes** each one costs.

PQ = compression

Split a D-dim vector into m sub-vectors; replace each with the nearest of 256 centroids — one byte apiece.

`768x4 B -> m B`

(m=96 => ~32x smaller)

ADC = cheap distance

Keep the query in full precision; only the database is quantised. Precompute query-to-centroid distances once, then sum lookups.

```
d(q,x)^2 = SUM_i
LUT_i[code_i]
```

IVF = prune + recall

K-means builds nlist cells; scan only the nprobe nearest. PQ encodes residuals, so quantisation error stays small.

`scanned = (nprobe/nlist).N`

In one line: IVF skips ~90% of vectors by clustering; PQ makes the survivors ~32x smaller and each distance a table lookup — recall is bought back with nprobe.

HNSW (Hierarchical Navigable Small World)

A fast and scalable graph-based index for approximate nearest neighbor search

What is HNSW?

HNSW is a hierarchical graph structure that enables fast approximate nearest neighbor (ANN) search in high-dimensional spaces.

Key ideas:

- **Multi-layer graph** (coarse to fine)
- **Small-world property** for short paths
- **Greedy search** across layers
- **Efficient insertions** and queries

Why it works

Small-world property: most nodes are reachable via a small number of hops.

Hierarchical structure: upper layers capture the global structure (few nodes), lower layers capture local details (many nodes).

Greedy search: makes good progress towards the query using local decisions.

Complexity

- **Search:** O(log N) ~ sub-linear (very fast in practice)
- **Insert:** O(M log N)
- **Space:** O(MN) (M = max connections per node)

N = number of elements, M = max neighbors per node

Hierarchical Structure

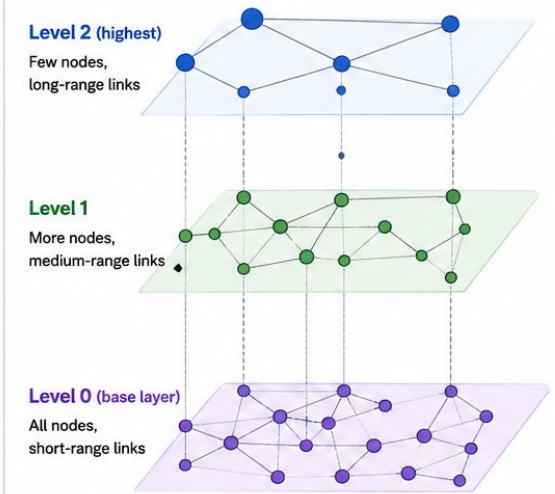
Top (coarse)

Bottom (fine)

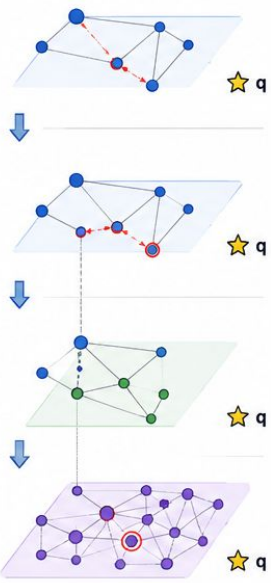
Level 2 (highest)
Few nodes, long-range links

Level 1
More nodes, medium-range links

Level 0 (base layer)
All nodes, short-range links



Search: how it works (for a query point q)



Greedy search + hierarchy = fast and accurate results!

Where is HNSW used?

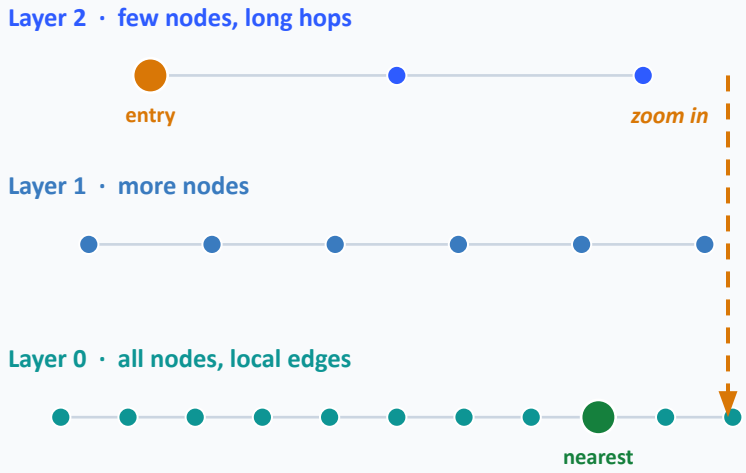
- Vector Search (semantic search)
- Recommendation Systems
- Image / Video Retrieval
- Bioinformatics & more

Why HNSW is popular?

- **High recall** with very low latency
- **Scales** to millions/billions of vectors
- **Dynamic:** supports insertions & deletions
- **Open-source** and widely used (FAISS, nmslib, Qdrant, Weaviate, Milvus, etc.)

HNSW: zoom in through layers

Like a road network: take **highways** to get near the destination fast, then drop to **local streets** for the exact address.



How the search runs

Start at the top entry node. Greedily hop to the neighbour closest to the query; when you can't get closer, **drop a layer** and repeat. Each descent zooms from coarse to local.

The ef dial (efSearch)

How many candidates to keep "in play" while exploring. **Low ef** = fast, may miss the true neighbour. **High ef** = wider search, higher recall, slower.

Same dial, new hat: ef is HNSW's recall–speed knob, exactly like nprobe is IVF's. Build-time M (edges per node) sets the graph's baseline quality.

efSearch: the candidate list that controls recall

The **width of the search at layer 0** — how many close candidates HNSW keeps in play before it answers.

Inside the layer-0 search

Upper layers just navigate (beam = 1). At layer 0 the search keeps the **ef** closest candidates, exploring neighbours until none beats the worst one kept.

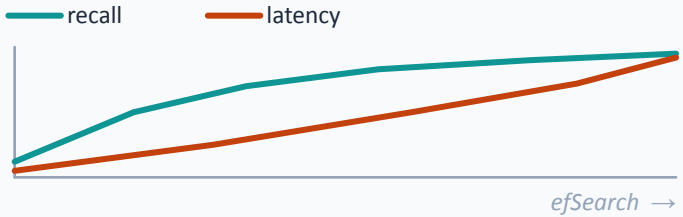
Candidate list (size ef = 4)

- **0.11 nearest**
- 0.14
- 0.17
- **0.22 worst kept**

new node d=0.13

0.13 < 0.22, so it is inserted and the worst is dropped.

Recall vs latency, as ef grows



Wider list more nodes visited → higher recall, slower.

ef ≥ k you return the top k from the ef candidates.

Query-time dial tune it per query — no rebuild.

HNSW: why search is ~ log(N), not N

Build a layered proximity graph so each hop covers more ground — a **skip list** lifted into high-dimensional space.

Skip-list intuition

A skip list adds express lanes over a sorted list. Each higher layer keeps a fraction of the nodes, so you leap exponentially far per hop.

```
P(node in layer L) = m^(-L)
```

Why it is logarithmic

Layers thin geometrically: the top holds O(1) nodes and each descent roughly halves the distance still to cover.

```
E[hops] = O(log N)
```

The two knobs

M = edges per node, fixed at build time (sets baseline recall + memory). **efSearch** = candidates kept in play, the query-time recall dial.

```
memory = O(N . M)
```

In one line: greedy descent through thinning layers turns a linear scan into a logarithmic walk — recall traded against ef, baseline fixed by M.

FAISS

Faiss is a library for efficient similarity search and clustering of dense vectors. It contains algorithms that search in sets of vectors of any size, up to ones that possibly do not fit in RAM.

Two main approaches:

1. **IndexFlatL2** : Exact L2 matching but faster than manual implementation
2. **IndexIVF** (Inverted File) : Cluster similar features together, only searches relevant clusters



ChromaDB: a database wrapped around vector search

FAISS is a **library** that finds nearest vectors. ChromaDB is the **database** that wraps it with everything a RAG app actually needs.

The raw-search problem

A bare similarity search returns **numbers**:

```
“vector #4471 is closest”
```

… but the LLM needs the **text**, and you need the **source** to filter & cite. With FAISS you track that ID→text map yourself — the classic *“numbers but no documents”* bug.

Chroma stores all three together

Vector	for the similarity math
Text	the actual chunk the LLM reads
Metadata	source, page, date — to filter & cite

The feature raw search can’t do: metadata filtering

```
where={"source": "policy.docx"}
```

Pre-filter before the search → higher precision AND a smaller, faster candidate set.

The trade-off

Under the hood: HNSW (one index, CPU, single node). Convenience now, a ceiling later — graduate to FAISS for billions or fine index control.

In one line: take an ANN library like FAISS, wrap it in a real database — text, metadata, persistence, filtering included.

Design Decisions of VectorDB

Two questions stack here:
(1) database or library? then **(2) which index?**

Question 1 - ChromaDB vs FAISS (vs managed)

1. **Prototype, course, app under ~a few million vectors, want metadata + persistence with minimal code**

Choose: ChromaDB.

Because: it stores vectors + original text + metadata together, persists to disk, and filters by metadata - a working semantic search service in ~15 lines. It runs FAISS-class indexes underneath anyway, so you rarely lose speed at this scale.

```
collection.query(query_embeddings=q, n_results=5, where={"source": "policy.docx"})
# WHY the `where` filter matters: users ask scoped questions ("in the HR policy…").
# Pre-filtering raises precision AND speeds search by shrinking the candidate set.
# FAISS alone can't do this — you'd bolt on your own metadata store.
```

2. Need raw speed / huge scale / fine index control / embedding inside another system

Choose: FAISS (the library).

Because: fastest and most tunable, scales to billions, full control over index type and compression. The cost: it's *not* a database - no persistence, no metadata, no text storage out of the box.

You must save/load the index file yourself and keep your own `id → text/metadata` map (losing that map is the classic "FAISS gave me numbers but I lost my documents" bug).

Question 2 - Which FAISS index? (corpus size is the main driver)

Branch 3D - Up to ~1M vectors, or you require 100% recall

Choose: `IndexFlatIP` (exact, brute force). **Because:** it compares the query to every vector — always finds the true nearest neighbors, no training, no tuning. **Don't trade away accuracy to solve a speed problem you don't have yet.**

Branch 3E - Millions of vectors, can tolerate a small recall loss, fits in RAM

Choose: `IVFFlat`. Knobs: `nlist ≈ √N`, `nprobe` = recall dial. **Because:** it clusters vectors and only searches the few clusters near the query, skipping ~90% of comparisons. **Needs training** (≥ ~30–40× `nlist` points or clustering is poor — that's the warning on small data). `nprobe=1` is fastest/lowest-recall; raising it trades latency for recall until, at `nprobe=nlist`, it degenerates back to exact.


menti.com
2182 6296

You are deploying a semantic search service for a dataset of 50 million document chunks. Your infrastructure has strict memory (RAM) constraints, but you can tolerate a slight drop in retrieval recall. Which indexing strategy is the most appropriate design choice?

A. IndexFlatIP

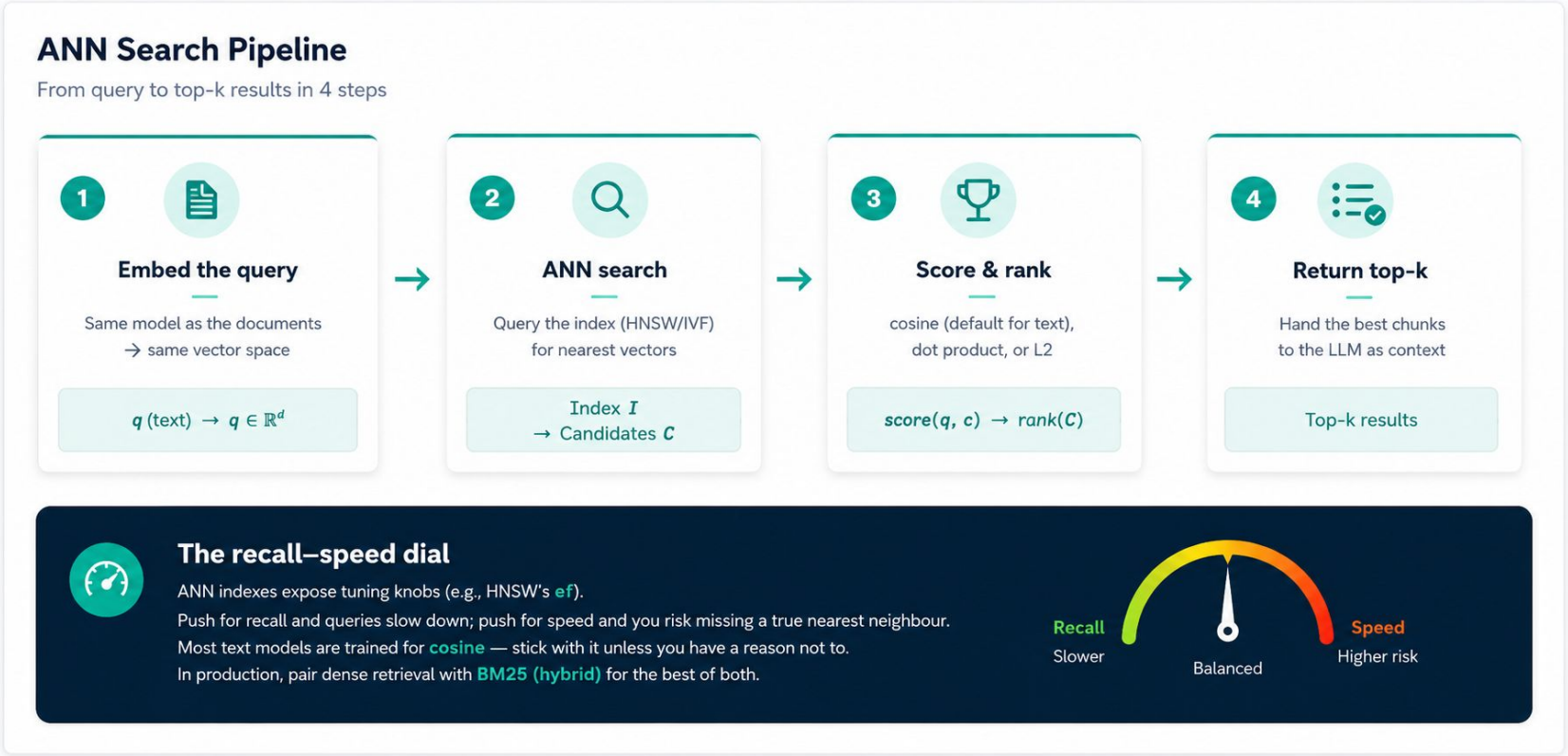
B. IVF + PQ (Inverted File + Product Quantization)

C. Exact Cross-Encoder Matching

D. HNSW (Hierarchical Navigable Small World)

Right answer: B
IVF reduces the search scope by clustering, and Product Quantization heavily compresses the vector memory footprint, directly addressing strict RAM limits for massive datasets.

The retrieval pipeline: Semantic search



A minimal end-to-end pipeline

```python
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2') # 384-dim, local

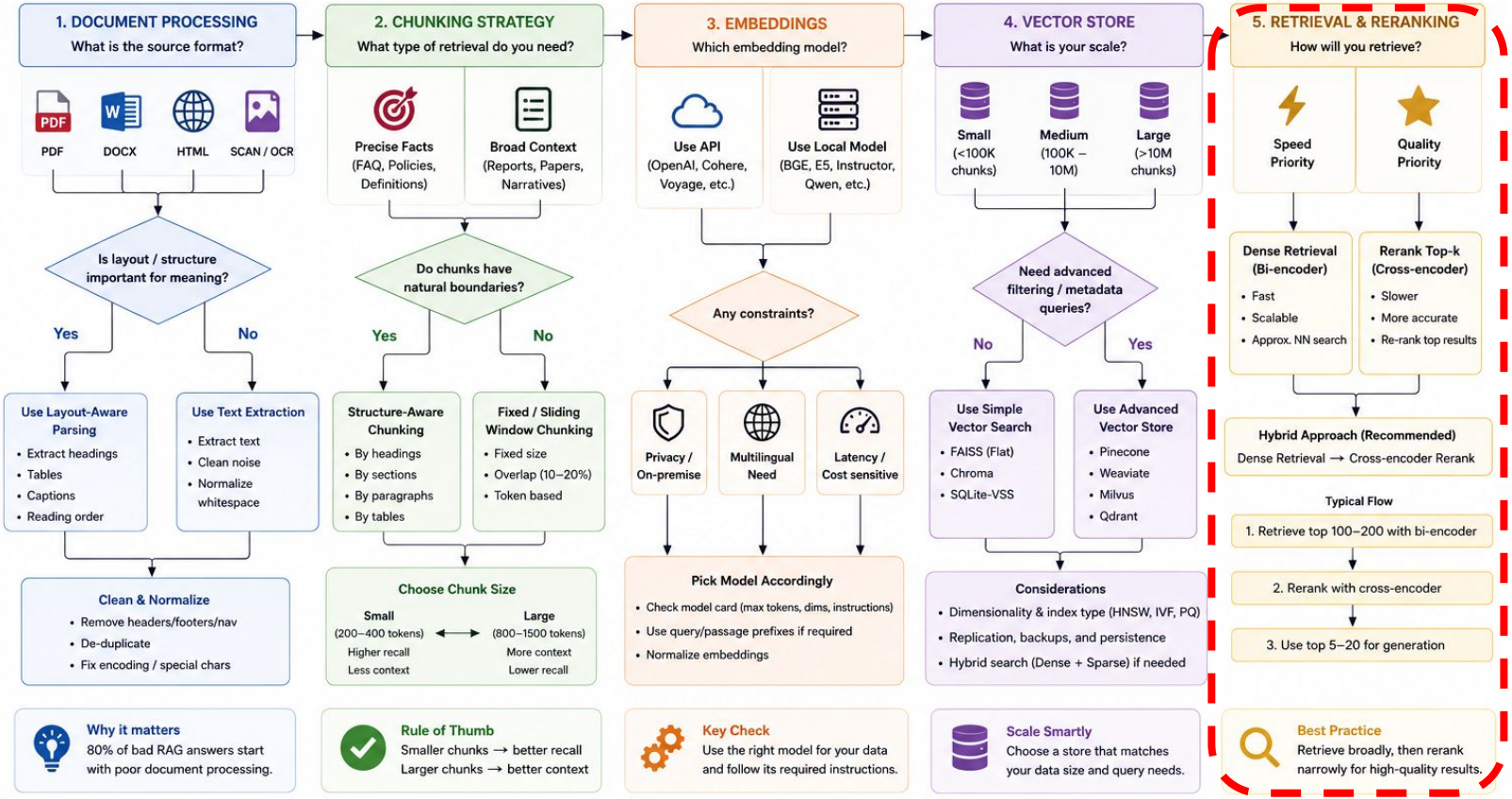
# 1) embed your chunks (documents already parsed + chunked)
emb = model.encode(chunks, normalize_embeddings=True)


# 2) build an index (cosine == inner product on L2-normed vectors)
index = faiss.IndexFlatIP(emb.shape[1])
index.add(np.asarray(emb, dtype='float32'))


# 3) embed the query with the SAME model, then search
q = model.encode(['how do I reduce API latency?'],
normalize_embeddings=True)
scores, ids = index.search(np.asarray(q, 'float32'), k=5)
top_chunks = [chunks[i] for i in ids[0]] # -> feed to the LLM
```

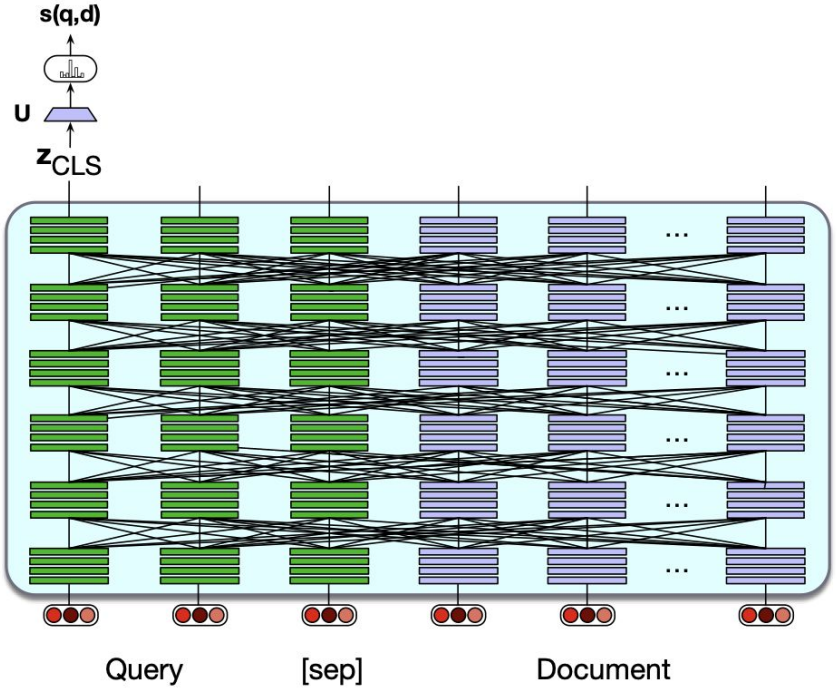
Design Choices in RAG Systems

Documents, Chunking & Vector Stores



Key Principle Good RAG is about making the right design choices at every step — clean data, meaningful chunks, appropriate embeddings, scalable storage, and effective retrieval.

Dense Retrieval (costlier but more accurate): Cross-Encoder



$$\mathbf{z} = \text{BERT}(\text{q}; [\text{SEP}]; \text{d})[\text{CLS}]$$

$$\text{score}(q, d) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$

Use a single encoder to jointly encode query and document and finetune to produce a relevance score with a linear layer over the CLS token. This is too compute-expensive to use except in rescoring

Bi-encoder vs Cross-encoder: the cost of accuracy

Same goal — score **(query, doc)** relevance — but **where you spend attention** decides the speed.

Bi-encoder: precompute

Encode query and doc separately, then score by a dot product. Doc vectors are built once, offline, and indexed.

```
score = E(q) . E(d)
```

docs encoded once

Cross-encoder: joint

Feed [q ; SEP ; d] through one transformer so every query token attends to every doc token. Far more accurate — nothing precomputable.

```
score = sig(W . BERT([q;d]))
```

Why two-stage

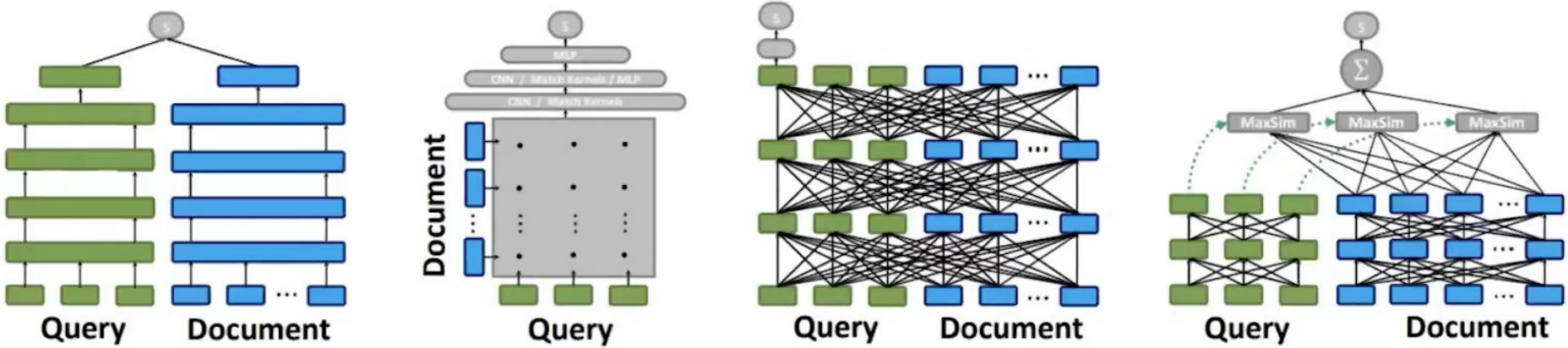
Cross-encoding all N docs is N forward passes — infeasible. Retrieve K cheaply, rerank only those K.

```
N passes -> K passes (K << N)
```

In one line: bi-encoders push work offline (cheap, approximate); cross-encoders pay per pair (precise, costly) — so you retrieve cheap and rerank dear.

ColBERT

- How do queries and documents interact? Going way beyond dot product.



(a) Representation-based Similarity
(e.g., DSSM, SNRM)

(b) Query-Document Interaction
(e.g., DRMM, KNRM, Conv-KNRM)

(c) All-to-all Interaction
(e.g., BERT)

(d) Late Interaction
(i.e., the proposed ColBERT)

ColBERT: late interaction (the middle ground)

Keep one embedding **per token**, not per document — then match tokens at query time.

Token-level embeddings

Store a vector per token, not per doc. These are precomputed and indexed offline, like a bi-encoder — no re-encoding at query time.

```
doc -> {e_1, e_2, ... e_n}
```

MaxSim operator

For each query token, take its best match among all doc tokens, then sum those maxima. Interaction happens late, after encoding.

```
score = SUM_i max_j (e_i . e_j)
```

The trade-off

Accuracy nears a cross-encoder at a fraction of the cost — but storing every token vector inflates the index.

```
storage ~ tokens, not docs
```

In one line: ColBERT defers interaction to query time via MaxSim — near cross-encoder quality with bi-encoder-style precompute, paid for in index size.

The two-stage pattern is industry standard

1 Retrieve

Bi-encoder + ANN index pulls top 100–200 candidates from millions — fast, scalable, approximate.

→

2 Rerank

Cross-encoder rescores each (query, candidate) pair jointly — slow but precise.

→

3 Generate

Top 5–20 reranked chunks go to the LLM as context.

Seen in: Google / Bing two-stage ranking · FAISS powering similarity search across Meta products

Three failures that silently wreck a RAG system

1

Empty chunks to prod

A scanned PDF has no text layer — pypdf returns empty strings silently. Always probe for a text layer; route to OCR if absent.

2

Half the recall, no warning

An asymmetric model used without query: / passage: prefixes. Read the model card.

3

Euclidean on a cosine model

Using the DB's default L2 metric on a model trained for cosine — quietly underperforms. Match the metric to training.

Each one fails quietly — no error, no crash, just worse answers. These are the bugs your participants will actually hit.


menti.com
2182 6296

An enterprise requires a retrieval pipeline that achieves accuracy close to a cross-encoder but strictly cannot afford the query-time latency of computing joint attention over all candidate document pairs. They are, however, willing to provision significantly more storage. Which architecture best satisfies these constraints?

A. A dual-stage Bi-encoder followed by Cross-encoder reranking

B. A standard Bi-encoder

C. BM25 Sparse Retrieval

D. ColBERT (Late Interaction)

Right answer: D
This model precomputes token-level embeddings, deferring interaction to a lightweight query-time MaxSim operation, which trades high storage costs for near cross-encoder accuracy without the heavy runtime compute.

ENTERPRISE RAG: TEXT + TABLES + SECURE NL-TO-SQL

1. DATA SOURCES

Unstructured (Text)

PDFs · Docs · Emails · ...

Structured (Tables)

Databases · Sheets · Data Warehouses · ...

2. INGESTION & PROCESSING

Text Pipeline

Extract → Clean → Chunk (Semantic) → Embed → Vector Store

Store: content, title, section, page, metadata, permissions

Table Pipeline

Summarization → Embed

Row / Cell Transformation → Embed

→ Vector Store (Table Summaries + Row Embeddings)

Store: table metadata, schema, data lineage, permissions

3. QUERY UNDERSTANDING & ROUTING

User Query

"Why did APAC revenue drop in Q3?"

Query Understanding

- Intent Classification (Text / Data / Hybrid)
- Entity Extraction
- Context Understanding

Router

→ Text RAG | Table (NL-to-SQL) | Hybrid (Both)

4. RETRIEVAL & EXECUTION

A. Text RAG (Vector Search)

Vector Search (Top-K)

Top Relevant Text Chunks ...

B. Table RAG (NL-to-SQL)

NL-to-SQL (LLM)

SQL Validation Layer (Security Checks)

Execute (Read-Only) (with RLS)

Structured Results ...

Hybrid Retrieval (Optional)

Combine text chunks + structured results

SECURE NL-TO-SQL ARCHITECTURE

User → Intent Classification → NL-to-SQL (LLM) → SQL Validation Layer (Guardrails) → Read-Only Database → Results → LLM (Answer Generation)

SQL Validation Layer (Guardrails)

- ✓ Allowed tables (Whitelist)
- ✓ Allowed columns (Whitelist)
- ✓ Allowed functions (Whitelist)
- ✓ Query type (SELECT only)
- ✓ No destructive queries (DROP, DELETE, UPDATE, INSERT, ALTER)
- ✓ Complexity & cost limits
- ✓ Result size limits

Row-Level Security (RLS) Enforced

Security Principles

- **Read-Only Access** — No write operations
- **Schema Whitelisting** — Only approved tables & columns exposed
- **Row-Level Security (RLS)** — Users see only their data
- **SQL Validation** — All queries checked before execution
- **Result Limits** — Prevent excessive data exposure
- **Audit Logs** — Log queries & access for monitoring

5. FUSION & GENERATION

Context Fusion — Merge & rank information from all sources

LLM Generation — Generate accurate, faithful, and cited responses

Final Answer — Answer with citations (text + data)

Citations

TABLE HANDLING STRATEGIES

1. Table Summarization

Convert table to natural language summary and embed.

Best for: Explanations, trends, insights

Year	Revenue
2023	100M
2024	120M

→ Revenue increased from $100M in 2023 to $120M in 2024.

2. Row-Level Embeddings

Convert each row to text and embed individually.

Best for: Lookups, filtering, entity-level retrieval

Region	Sales	Profit
APAC	100M	20M
EMEA	80M	15M

→ Region APAC had sales of 100M and profit of 20M.

3. Hybrid (Vector + SQL)

Store summaries/rows in vector DB and keep raw tables in SQL.

Best for: Analytics, aggregations, precise numeric queries

Vector DB (Summaries & Rows) + SQL DB (Raw Tables)

WHY HYBRID APPROACH?

- ✓ Vectors are great for meaning, not for exact numbers.
- ✓ SQL is reliable for calculations, aggregations, rankings.
- ✓ Combining both gives accuracy + explainability.
- ✓ Secure NL-to-SQL prevents data leakage and misuse.

KEY TAKEAWAYS

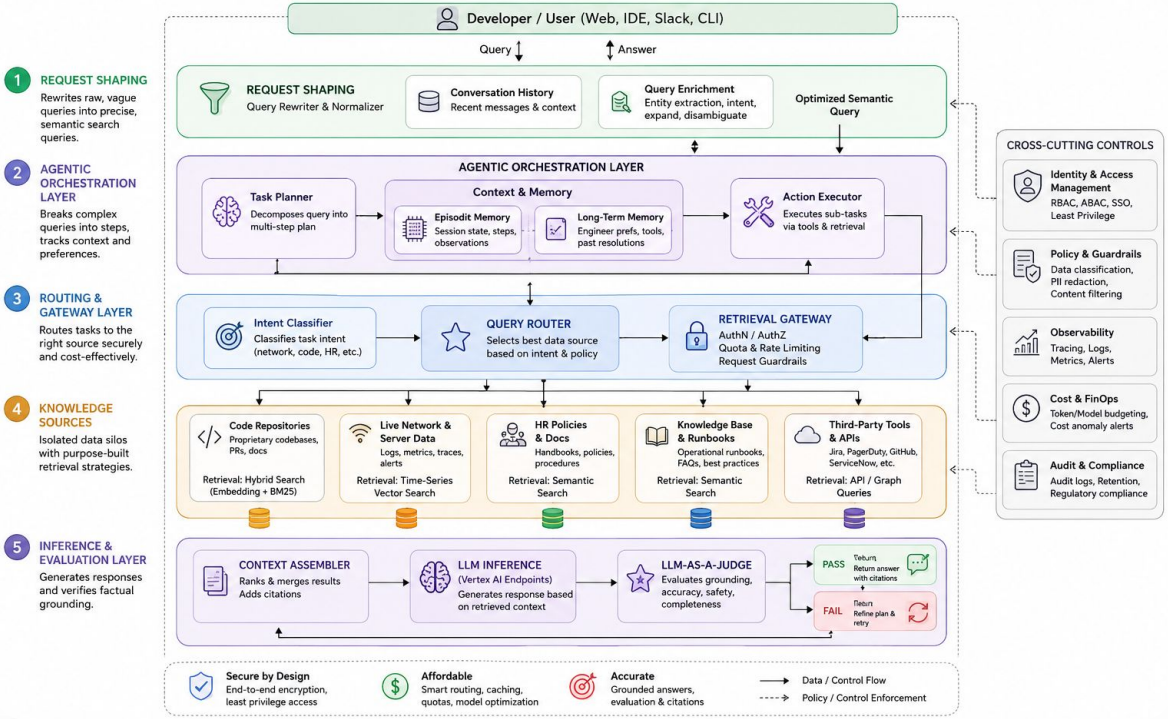
- Use the right tool for the right data type.
- Separate retrieval for text and structured data.
- Always validate and secure NL-to-SQL.
- Enforce least privilege and data governance.
- Provide accurate, trustworthy, and auditable answers.

RAG in Production: *Transition to Agentic RAG*

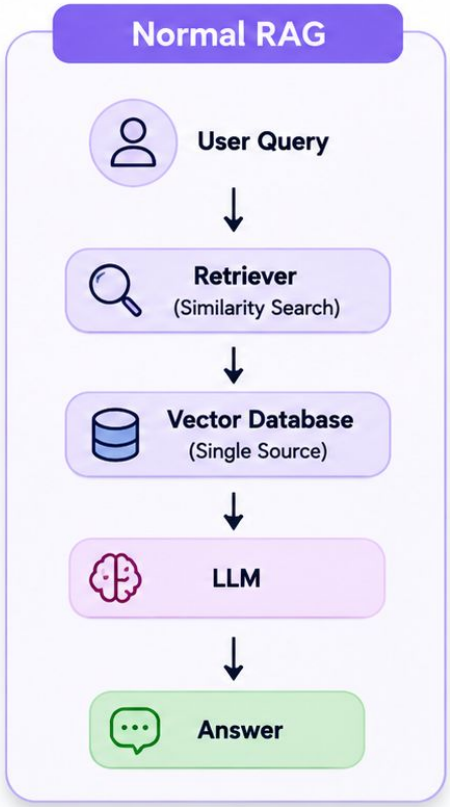
The Enterprise DevOps & Support Agent

The Goal: Build an internal AI agent for software engineering teams.

Key Requirements: Troubleshoot incidents, query codebase, answer HR queries, and ensure security.



Production Rag Systems (What's different?)



What's New in Production RAG?

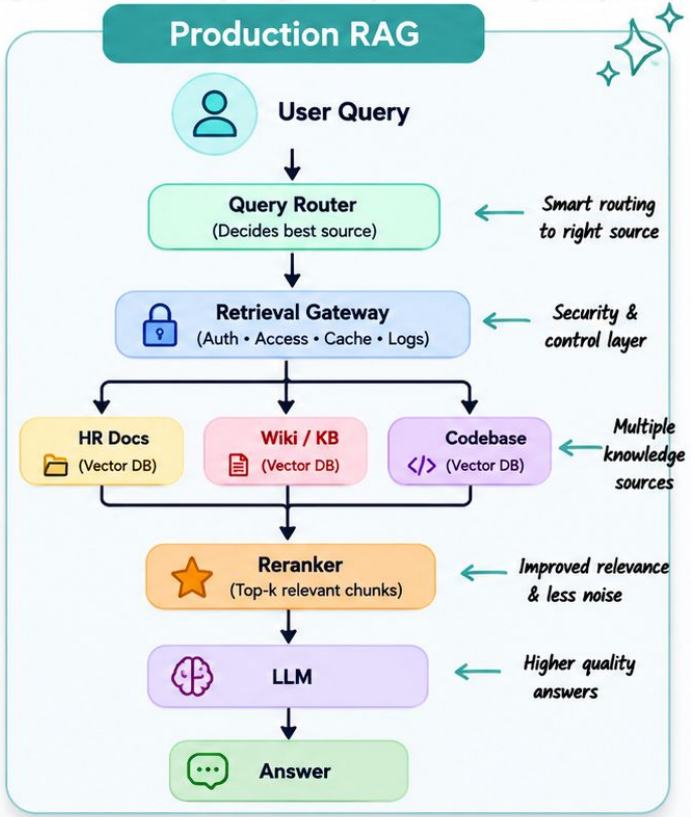
Multiple Data Sources
Route queries to the right source

Retrieval Gates
Auth, access control, caching, logging, monitoring

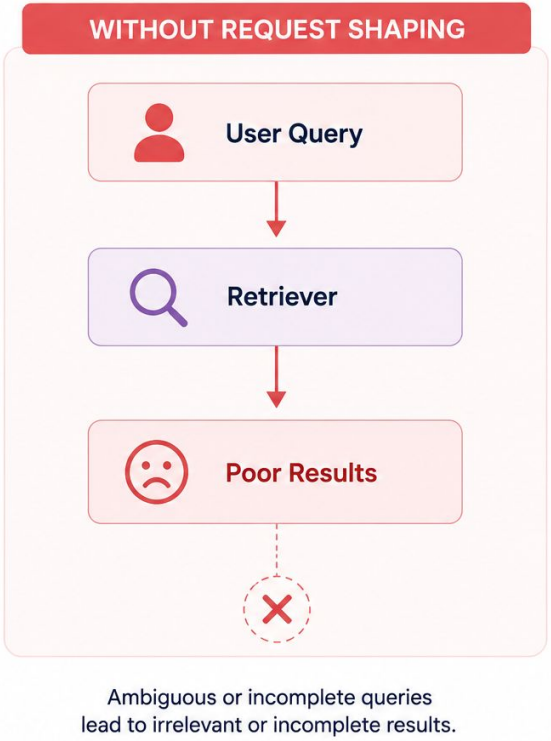
Request Shaping
Rewrite, expand, filter queries before retrieval

Better Retrieval Quality
Reranking, deduplication, context compression

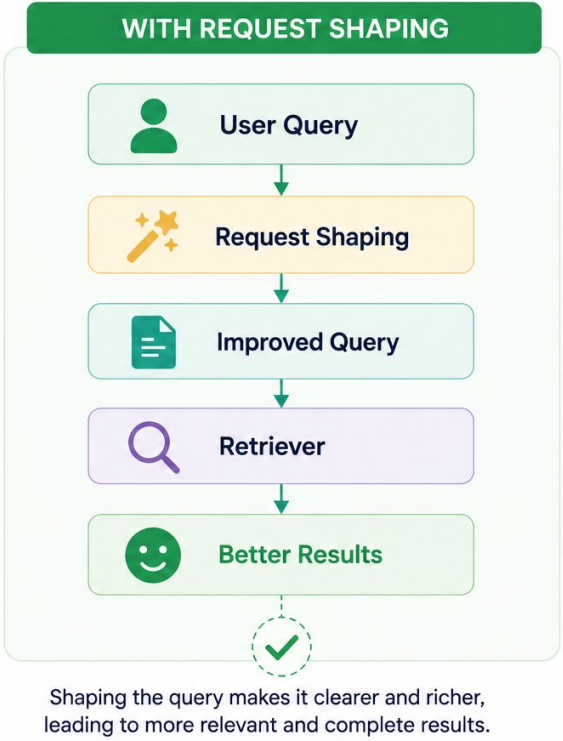
Production Concerns
Scalability, latency, reliability, observability



Request Shaping



Ambiguous or incomplete queries lead to irrelevant or incomplete results.

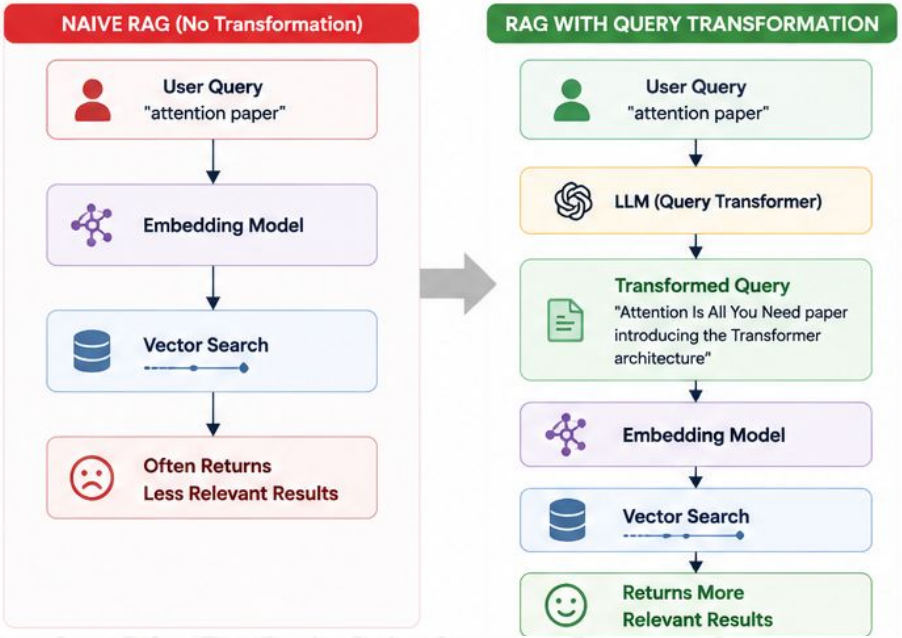


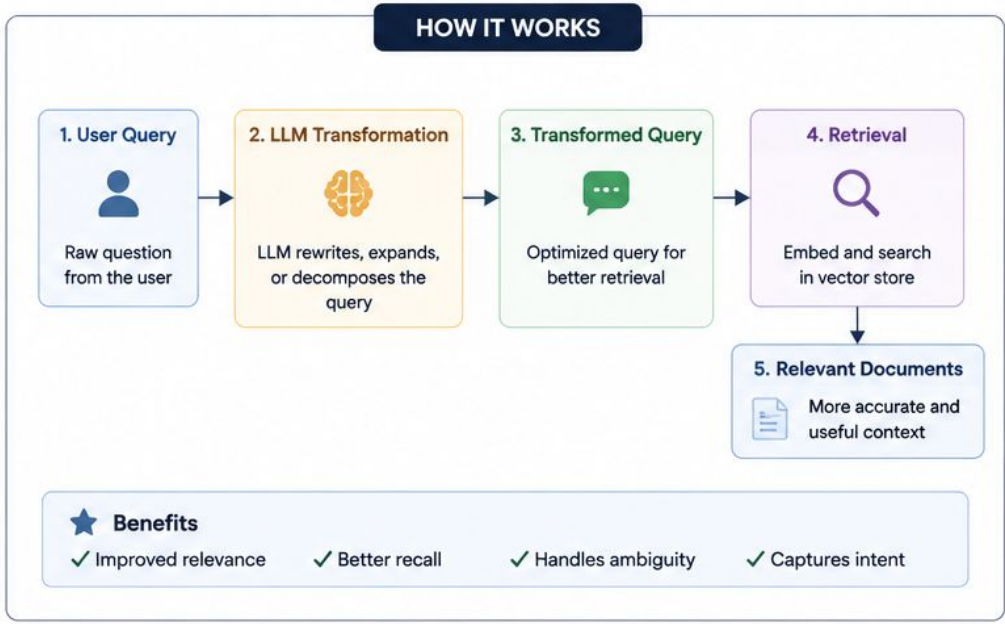
Shaping the query makes it clearer and richer, leading to more relevant and complete results.

- Motivation : Requests made by users are not always precise and usable.
- Example: *Teach me attention*. This prompt doesn't specify what attention really means.
- **Request shaping transforms a user query before retrieval to improve relevance, precision, and retrieval quality.**

QUERY TRANSFORMATION IN RAG

Using an LLM to transform the user query before retrieval to improve relevance and results





Example of Request Shaping in Project Context

Original User Query	Transformed / Shaped Request
“Server down.”	**“Find active incident reports and server logs for the past 60 minutes.”**
“Car leasing policy”	**“Information about Employee benefit: Car-Leasing policy”**
“Latest bugs assigned”	**“Fetch top-10 bugs assigned to the user”**

Request Shaping in RAG – How it Works Internally

Request shaping improves the original user query to retrieve better, more relevant context.

1 User Query

User asks a question in natural language.

Example:
"Why did APAC revenue drop in Q3?"

2 Request Shaping (LLM Call #1)

LLM (Request Shaper)

Rewrites / expands / decomposes the query to improve retrieval.

Techniques used:
- Query rewriting
- Expansion (synonyms, keywords)
- Decomposition (sub-questions)
- Clarification (add filters, context)

Output: Improved Query
"Reasons for APAC revenue decline in Q3, factors affecting sales, APAC Q3 performance drop"

3 Retrieval

Vector Search (Bi-encoder)

Search using improved query

Vector Database (Document Chunks)

Top-K Relevant (Documents / Chunks

4 Answer Generation (LLM Call #2)

LLM (Answer Generator)

Uses retrieved context + original query to generate the final answer.

Inputs to this LLM:
- Original user query
- Retrieved context
- (Optional) Improved query

Final Answer
Accurate, contextual, well-grounded response.

Is it the Same LLM?

It can be the **SAME** LLM or **DIFFERENT LLMs.**

Common approaches:
- **Same LLM:** Simpler to manage, lower cost (if using one model).
- **Different LLMs:** Use a small/ cheap model for shaping, and a larger model for answer generation.

Why Multiple LLM Calls?

- Specialized tasks produce better results.
- Shaping improves retrieval quality.
- Final answer is more accurate and grounded.

Inside the Request Shaping LLM (What's happening?)

Input User Query	Understand Intent & Context	Expand & Enrich	Rewrite / Decompose	Improved Query (Output)
Why did APAC revenue drop in Q3?	Identify the core intent, entities, time period, metrics, region, etc.	Add synonyms, related terms, metrics, and business context.	Rephrase for clarity or break into sub-questions (if complex).	Optimized query for better retrieval from vector database.

Example

Original Query:
Why did APAC revenue drop in Q3?

Improved Query (after shaping):
"Reasons for APAC revenue decline in Q3, APAC Q3 sales performance drop factors, impact of market, demand, churn in APAC Q3"

Key Takeaway:
Request shaping is an important first **LLM call** that transforms the user's question into a better search query. It leads to higher quality retrieval, which ultimately results in more accurate and trustworthy answers.

LLM Call #1 Request Shaping → Retrieval (Non-LLM) → LLM Call #2 Answer Generation = Better Answer

COMMON QUERY TRANSFORMATION TECHNIQUES & EXAMPLES

1 Query Rewriting

User Query (Original)

attention paper

↓

Transformed Query (Rewritten)

"Attention Is All You Need paper introducing the Transformer architecture"

Why it helps retrieval

- Resolves ambiguity and vagueness
- Adds missing context and specificity
- Improves lexical + semantic match
- Increases precision & recall

Used in

RAG-Fusion, Azure AI Search, LangChain SelfQueryRetriever

How it works

An LLM rewrites the user query to be clearer, more specific, and better aligned with how relevant documents are expressed.

2 Follow-up Question Transformation

Conversation

User: What can I do in Italy?

User: What type of food is there?

↓

Transformed Query

"What types of food are popular in Italy?"

Why it helps retrieval

- Resolves coreferences (it, there, they)
- Converts follow-ups to standalone form
- Maintains conversation context
- Improves retrieval relevance

Used in

Conversational RAG, ChatGPT Retrieval, LangChain Chat History Aware Retriever

How it works

The follow-up question is converted into a standalone question using the prior conversation context.

3 Multi-Query Retrieval

User Query

Who won a championship more recently, the Red Sox or the Patriots?

↓

Generated Queries

- When did the Red Sox last win a championship?
- When did the Patriots last win a championship?
- Which team won a championship more recently, Red Sox or Patriots?
- Red Sox last championship year
- Patriots last championship year

... (and more)

Why it helps retrieval

- Captures multiple aspects & phrasings
- Expands recall across different angles
- Reduces embedding miss
- Improves answer completeness

Used in

RAG-Fusion, LangChain MultiQueryRetriever, Promptagator, HyDE (Hypothetical Document Embeddings)

How it works

An LLM generates several semantically diverse versions of the query. The system retrieves documents for all queries and merges (union + dedup) the results.

4 Step-Back Prompting

User Query

How does BERT use attention?

↓

Step-Back (Broader) Query

How does attention work in Transformer architectures?

↓

Retrieve Foundational Context



↓

Answer Original Query with Stronger Context

Why it helps retrieval

- Moves to a higher abstraction level
- Retrieves foundational knowledge
- Bridges gaps in implicit assumptions
- Improves reasoning & answer quality

Used in

Anthropic (Step-Back Prompting), LangChain, Advanced RAG Systems

How it works

The system first retrieves high-level, conceptual information related to the query, then uses it to better answer the original, more specific question.

Where it fits in the RAG Pipeline (Request Shaping Layer)

User Query → Query Transformation (These Techniques) → Embedding → Retrieval (Vector Search) → Reranking → Generation (LLM Answer)

Key Takeaway

Better queries lead to better retrieval, which leads to better answers.

Query Transformations: Rewriting

What it is: The process of using an LLM to rewrite ambiguous, vague, or shorthand queries into clearer, highly specific search statements.

When to use it: Essential for support channels (Slack, Teams) where fragmented sentences, error codes, or jargon hinder vector database matching.

Project Example: DevOps Agent

USER QUERY (ORIGINAL)

"OOMKilled"

—

TRANSFORMED QUERY (REWRITTEN)

What causes the OOMKilled container exit code in Kubernetes, and what are the remediation steps?

The Impact: Instead of returning random documents containing "OOM", the agent retrieves specific internal runbooks for memory allocation and container scaling.

Query Transformations: Follow-up Question

What it is: Converting contextual follow-up questions into complete, standalone queries by injecting the history of the current conversation.

When to use it: During active, multi-turn troubleshooting or policy inquiries where the user naturally relies on pronouns ("it," "they," "that server").

Project Example: HR / Support Agent

CONVERSATION HISTORY

User: "What is the standard parental leave policy?"

Agent: [Returns standard policy details]

***User:* "Does it apply to contractors?"**



TRANSFORMED QUERY

```
"Does the standard parental leave
policy apply to contract
employees?"
```

The Impact: Without this step, the retriever searches for "Does it apply to contractors?" and fails. The standalone query ensures high-accuracy retrieval.

Query Transformations: Multi-Query Retrieval

What it is: Generating multiple, distinct related queries from a single complex question and retrieving documents for all of them in parallel.

When to use it: When a prompt spans multiple microservices, compares entities, or requires joining info from separate documents.

Project Example: DevOps Agent

USER QUERY

"Did the latency spike happen in both the Mumbai and US-East Vertex AI deployments?"



GENERATED QUERIES

Q1: `What are the recent latency metrics and error logs for the Mumbai Vertex AI deployment?`

Q2: `What are the recent latency metrics and error logs for the US-East Vertex AI deployment?`

The Impact: Guarantees the LLM gets necessary context for *both* deployments before comparison, avoiding regional bias.

Query Transformations: Step-Back Prompting

What it is: Creating a broader, high-level question to retrieve foundational context *before* attempting to answer a hyper-specific user query.

When to use it: When an engineer's query is so deep in the weeds that it misses underlying architectural rules, leading to "no results" or hallucinations.

Project Example: DevOps Agent

USER QUERY

"Why can't I pull the switch association data for the rogue units on Floor 3?"

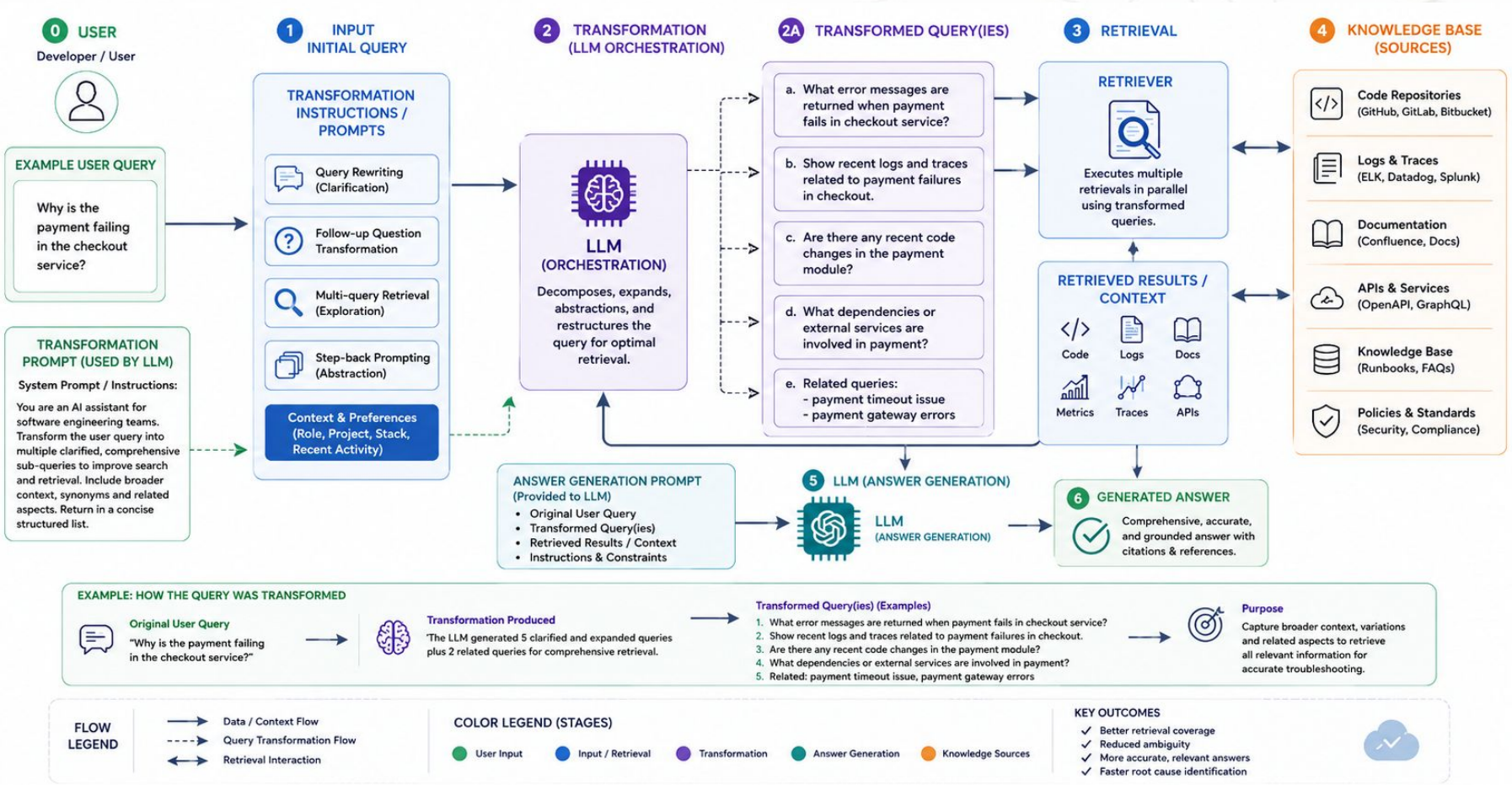


STEP-BACK (BROADER) QUERY

`"What is the overall architecture of our network troubleshooting system, and what data is collected from reporting access points versus rogue units?"`

The Impact: The step-back query retrieves the foundational rule: *switch association data is only available for reporting access points, not rogue units.* This allows the agent to explain the architectural limitation.

How will this be achieved?




menti.com
2182 6296

A user repeatedly encounters 'no results' or hallucinated answers when asking highly specific, deep-in-the-weeds technical questions (e.g., 'Why can't I pull the switch association data for the rogue units on Floor 3?'). The vector database contains the underlying architectural rules that explain this limitation, but it lacks the highly specific keywords from the user's prompt. Which query transformation design pattern is specifically built to mitigate this failure mode?

A. Follow-up Question Transformation

B. Semantic Routing

C. Step-Back Prompting

D. Multi-Query Retrieval

Right answer: C
Step-Back Prompting uses an LLM to abstract a hyper-specific query into a broader, foundational question (e.g., 'What is the overall architecture of our network troubleshooting system?'), allowing the system to retrieve the underlying rules before answering the specific edge case.

The Need for Dynamic Decisions for User Queries

The Immediate Problem

We now have four powerful ways to transform a query (Rewriting, Follow-ups, Multi-Query, Step-Back). But how does the system know ***which*** technique to apply to a specific user prompt?

The Broader RAG Challenge

As our DevOps Agent scales, this decision-making dilemma appears everywhere:

- *Which transformation technique should I use?*
- *Should I search the HR vector database or the GitHub code repository?*
- *Do I need a fast, cheap LLM for this, or a highly capable, expensive reasoning model?*

The Solution: Query Routing

Hardcoding `if/else` statements for every scenario creates a brittle system. We need an intelligent layer that dynamically routes requests to the optimal workflow, tool, or data source.

Query Routing: The Traffic Cop of Production RAG

What it is:

Query routing directs a user query to the most relevant knowledge source, workflow, or model before execution.

The "So What?" (Production Impact)

If a user asks a simple question, searching the entire company's infrastructure (HR, Codebase, Logs, Wiki) is slow, expensive, and increases the chance of the LLM hallucinating by pulling irrelevant data.

Routing isolates the search to only the necessary domains.

The Goal:

Maximize accuracy while minimizing latency and API costs.

Route each user query to the right data, tool, model and strategy for accurate, efficient and secure answers.

1. WHAT IS QUERY ROUTING?

Decide which retrieval strategy, index, **datasource, tool, workflow or model** should handle the user's request.

Instead of:

Question → Vector DB

Production RAG does:

Question → Query Router → Text RAG | SQL Engine | Graph DB | Tool / API | ...

2. HOW QUERY ROUTING WORKS (INTERNAL FLOW)

1 Query Understanding
- Parse
- Normalize
- Detect language, entities, time, metrics, filters

2 Multi-Dimensional Classification
- Intent Classification (Informational / Analytical / Transactional / Lookup / Comparison)
- Domain Classification (HR / Finance / Legal / Engg / Sales / ...)
- Data-Type Classification (Text / Table / Graph / Image / Code / Code / Unstructured)

3 Query Router (Strategy Selection)
- Choose best route(s)
- Single or multi-route
- Select index, tool, model & params

4 Retrieval Execution
- Execute on selected routes in parallel
- Apply filters, permissions
- Dynamic Top-K

5 Post-Retrieval Processing
- Fusion (merge results)
- Deduplicate
- Cross-encoder reranking

6 LLM Answer Generation
- Provide final answer with citations
- Grounded, safe & accurate

ROUTING DESTINATIONS (EXAMPLES)

- **Vector RAG** (Documents)
- **SQL Engine** (Tables / Data)
- **Knowledge Graph** (Entities / Relations)
- **Search Engine** (Web / Enterprise)
- **Tool / API** (Weather / Stocks / Calculator / ...)
- **Code Interpreter** (Python / Analysis)
- ...

3. EXAMPLE ROUTING DECISIONS

- "What is our PTO policy?" → Route: **Policy Documents** (Vector RAG – HR KB)
- "What was APAC revenue in Q3?" → Route: **SQL Engine** (Finance Data Warehouse)
- "Who reports to the VP of Engineering?" → Route: **Knowledge Graph** (Org Graph)
- "What is the weather in London?" → Route: **External Tool** (Weather API)
- "Why did APAC revenue drop?" → Route: **Multi-Route** (SQL + Documents) (Numbers + Commentary)

4. CORE ROUTING TECHNIQUES (HOW THE ROUTER DECIDES)

1. Rule-Based Routing
- If / else rules, keywords, patterns, metadata
- Fast, cheap, explainable
- Brittle at scale

```
if "revenue" in query → SQL
if "policy" in query → VectorDB
```

2. Embedding-Based Routing
- Embed query
- Compare with route embeddings
- Choose highest similarity

Similarity Scores (Example)

Finance	0.92
HR	0.31
Legal	0.27
...	

3. LLM Router
- Give options to LLM
- LLM selects best route(s)
- Handles nuance & context

Prompt: Choose best route
1. HR 2. Finance 3. Legal
4. SQL 5. Search 6. Tool

LLM Output: {"route": "Finance"}

4. Hierarchical Routing
- Reduce large search space
- Route step-by-step (Coarse → Fine)

Level 1: Business Unit (Finance)
↓
Level 2: Department (Revenue)
↓
Level 3: Datasource (Rev Analytics)
↓
Level 4: Destination (SQL Engine)

5. Multi-Route Retrieval
- Some queries need multiple sources
- Retrieve in parallel and fuse

Example:
Why did APAC revenue drop?
SQL (Numbers) + Documents (Commentary)
→ Fused Results

5. WHY ROUTING MATTERS

- ✓ Right data for the right question
- ✓ Better accuracy & grounding
- ✓ Lower cost (use right model / tool)
- ✓ Faster responses
- ✓ Safe, secure & governed access
- ✓ Scales across many data sources

6. ADVANCED PRODUCTION TECHNIQUES

A. Retrieval Confidence Routing
Check confidence after retrieval

Top Result Score = 0.42 (Low Confidence)

Options:
- Search another source
- Increase Top-K
- Ask clarification
- Escalate to human

B. Cost-Aware Routing
Route based on cost/complexity

Simple Query → Small
"What is PTO?" → K = 5
→ Cheap

Complex Query
"Analyze revenue decline" trends across regions"
- Large Model → K = 50
→ More Expensive

C. Dynamic Top-K Routing
Adjust Top-K based on query type

- Fact / Lookup K = 3 – 5
- General Q&A K = 10 – 15
- Complex / Analytical K = 30 – 50

D. Query Decomposition Routing
Break complex query into sub-queries

Example Query:
Compare APAC and EMEA revenue and explain major differences.

- Subquery 1 APAC revenue
- Subquery 2 EMEA revenue
- Subquery 3 Explanation / Commentary

Route each → Execute → Merge

E. Agentic / Planner Routing
LLM plans multi-step retrieval

Planner Output (Plan):
1. Query SQL for revenue data
2. Query Docs for narrative
3. Query Graph for org changes
4. Synthesize & answer

Execute plan → Merge results

7. KEY TAKEAWAYS

- Routing is often more important than the vector database.
- Not every query should go to vector search.
- Use the right strategy, data and model for each query.
- Good routing = better answers, lower cost, and happier users.

TECHNICAL ENABLERS

- **Metadata & Catalog** (Domains, Ownership, Freshness, Schema)
- **Access Control** (Row/Column Level Permissions)
- **Observability** (Routing Logs, Metrics, Traces)
- **Feedback Loop** (Human / LLM Feedback to improve routing)
- **Evaluation** (Per-route metrics: Accuracy, Cost, Latency)

COMMON ROUTING INPUT FEATURES

Query text, entities, time, metrics, filters, past conversation, user role, location, device, history, intent signals, embeddings.

FAILURES TO AVOID

- Wrong route → irrelevant results
- Over-routing → high latency & cost
- Under-routing → incomplete answers
- No fallback → failed user experience

OUTCOME

Right information, from the right place, at the right cost.

Routing Strategies: From Simple to Smart

Rule-Based (Logical)

Hard Coded logic or Regex. Fast and deterministic.

e.g., If query contains "PTO" or "leave", route to HR.

Embedding-Based

Semantic Router using vector similarity against route profiles. Fast, scalable, and offline-capable.

LLM-Based

LLM classification for JSON decisions. Highly accurate for ambiguity, but adds latency/costs.

Classifier-Based Routing

Train a dedicated intent classifier for core business domains:

- HR / Legal
- Research / Code

Hybrid Routing (The Pro Approach)

Workflow: Rule → Embedding → LLM

Logic:

Only trigger the expensive LLM router if the fast Embedding router has a low confidence score.

Rule-Based (Logical) Routing

What it is & How it works

The most fundamental routing technique. It relies on hard coded logic, keywords, or Regular Expressions (Regex) to map a raw query string against `if/else` conditions before AI processing.

Project Example: DevOps Agent

Logic:

```
IF query.contains(["PTO", "leave", "payroll", "manager"]): route_to = "HR_Database"
```

User Query:

"How many days of leave do I have left?"

The Result:

Instant route to HR docs. Cost: **$0.00** | Latency: **~1ms**

The Catch: Fragility

Highly brittle system. It lacks semantic understanding and fails if exact keywords are missing.

USER TYPES:

"I need to take time off"

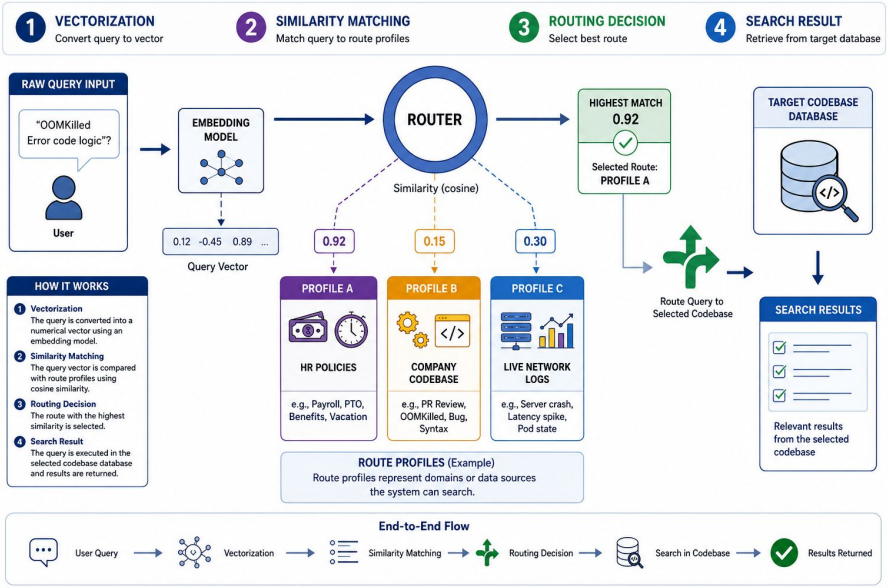
SYSTEM STATUS:

✘ **REGEX FAILS**

Missing exact keyword "leave" or "PTO".

Deterministic • Zero Cost • Extremely Low Latency • Low Generalization

Embedding based routing



What it is:

Transforms queries into vectors for similarity checks against "route profiles". Deterministic, fast (no LLMs), and works offline.

How it is done:

Create "profiles" (20-50 sample questions) for each database. Real queries are embedded and matched via cosine similarity.

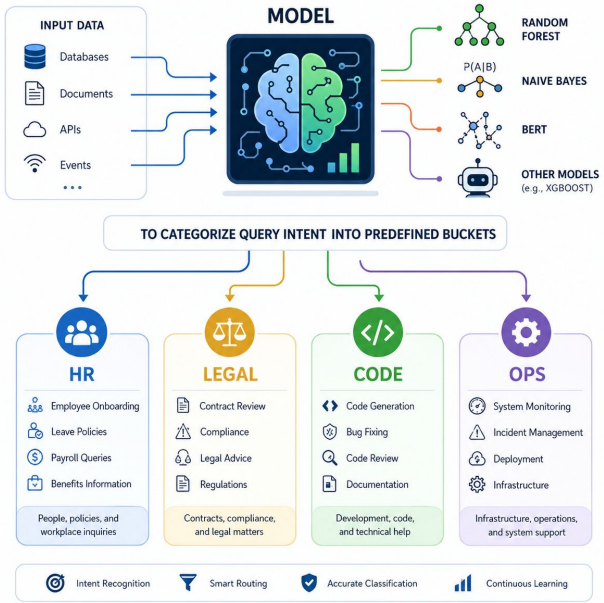
Project Example: DevOps Agent

User Query: "I'm feeling burnt out and need a break next Tuesday."

Route Profiles: HR_Profile (benefits, time off) vs. Code_Profile (bugs, PRs).

Result: Semantic match favors **HR_Profile** cluster. Routed without a heavy LLM.

Classifier-Based Routing: Learning Intent



What it is

Uses a trained, lightweight ML model (e.g., Random Forest, Naive Bayes, small BERT) to categorize query intent into predefined buckets like HR, Legal, or Ops.

How it is done

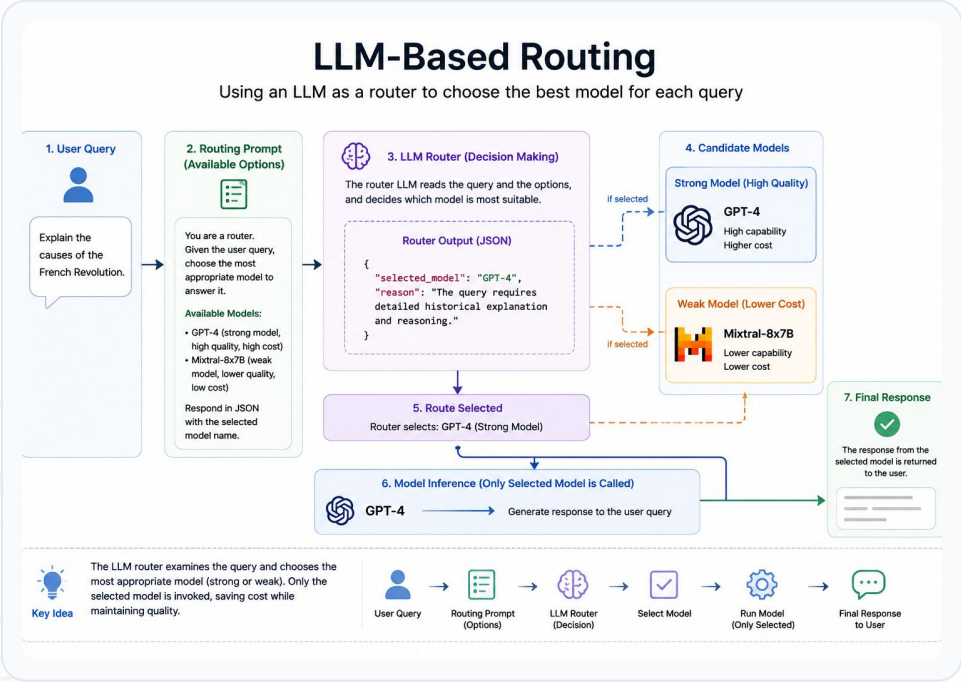
The classifier is trained on historical queries mapped to correct labels. At runtime, it ingests queries and outputs probability scores for each category.

Project Example: DevOps Agent

Query: "My compensation seems wrong this month." | **Result:** Recognizes semantic pattern

`Intent: HR (95% confidence)`

LLM based routing



Concept & Implementation

What it is: Simple queries use cheaper models; complex ones use stronger models. Ask LLM to reason and output structured JSON.

How it is done: A prompt with query and route descriptions asks: *"Analyze this query and return a JSON object with 'target_database' and 'reasoning'."*

Project Example: DevOps Agent

Query: "The auth token is expiring too fast. Is this a new security policy thing or a bug?"

Result: LLM recognizes technical execution over policy, outputting `{"route": "Codebase_DB"}`.

The Catch: Balances quality vs. cost dynamically, but adds latency and API cost to the pipeline.

Hybrid routing

What it is: Hybrid routing combines multiple strategies to leverage individual strengths while mitigating weaknesses.

How it is done: Routers are structured sequentially as a **waterfall**. Moving to the next stage occurs only upon low confidence.

1. Rule-Based — 2. Embedding-Based — 3. LLM-Based Sequential Workflow

Project Example: DevOps Agent

User Query: "Pipeline failed."

Step 1: Rule-Based

Result: **FAIL**

No obvious keywords found in query.

Step 2: Embedding

Result: **50% Confidence**

Ambiguous (CI/CD vs Sales). Execution falls back to next step.

Step 3: LLM-Based

Result: **SUCCESS**

Reviews query + Metadata (Role: DevOps). Accurately routes to Infrastructure_Logs.

Cost efficiency: Only paid for expensive LLM when cheaper methods failed.

Application 1: Data Source Routing

General Use: Selecting the correct vector database, SQL database, or API based on the user's intent.

Project Example (DevOps Agent): Since RAG outputs depend on the data-corpus it accesses, different questions must refer to different corpora.

HR & POLICY

User Query:
"What is the maternity leave policy?"

Route:
Retrieve from the **HR Docs Vector DB**.

TECHNICAL ASSETS

User Query:
"How does the auth module handle token expiry?"

Route:
Retrieve from the **Company Codebase Vector DB**.

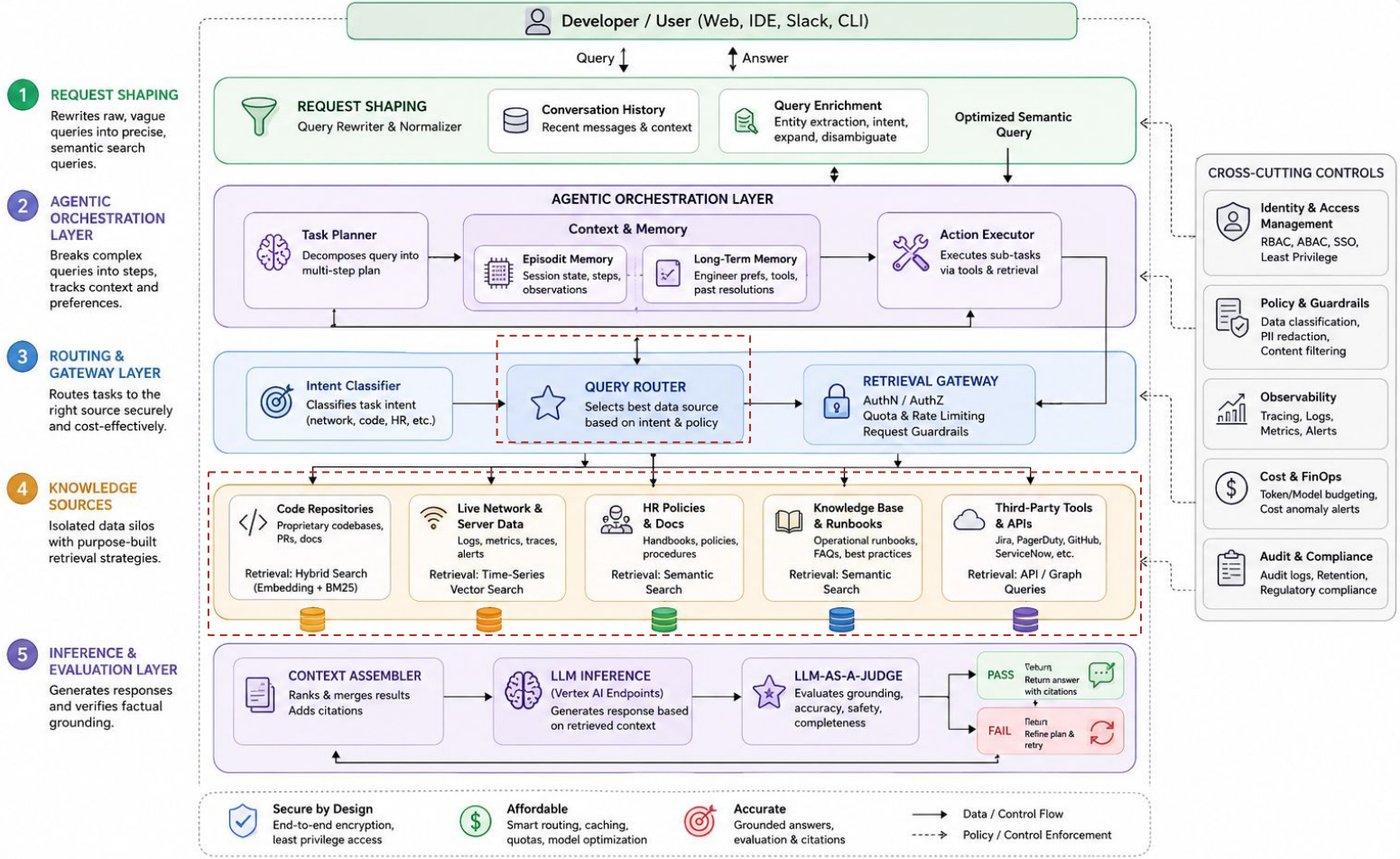
LIVE MONITORING

User Query:
"Why did the payment container crash 5 min ago?"

Route:
Trigger an API call to **Live Splunk/Datadog Logs**.
(Bypass Vector DB entirely)

Intent-based routing ensures the RAG pipeline targets the most relevant high-fidelity data source for every unique request.

Application 1: Data Source Routing - Where in Project?



Application 2: Model & Workflow Routing

General Use: Routing isn't just for finding documents; it is used to dynamically select *which LLM* or *which pipeline* to use, balancing response quality and inference cost dynamically for each query.

SIMPLE EXTRACTION

User Query:

"Format this server log into a JSON block."

Route to Model:

Direct to a smaller, faster, cheaper model (e.g., **Llama-3-8B** or **Haiku**).

Lowers latency and avoids wasting expensive model capacity.

COMPLEX REASONING

User Query:

"Review this 500-line Python PR, check it against our security guidelines, and draft a summary."

Route to Workflow:

Direct to a premium reasoning model (e.g., **GPT-4** or **Opus**).

Triggers a multi-step **Agentic Orchestration** workflow.

Simple queries are sent to cheaper models while complex queries use stronger models to optimize cost and quality.

Application 3: Prompt & Index Routing

General Use: Deciding *how* to search the data or *how* to prompt the LLM once the data is found.

INDEX ROUTING EXAMPLE

User Query:

"Find the specific error code in the network manual."

Route to Index:

Route to a **Sparse Index (BM25/Keyword Search)**.

Exact matches matter more than semantic meaning here.

PROMPT ROUTING EXAMPLE

User Query:

"Summarize the architectural changes made to the database last quarter."

Route to Prompt:

Route to a **Summarization Prompt Template**.

Instructs the LLM to aggregate rather than extract a single fact.

Project Example: DevOps Agent Case Study


menti.com
2182 6296

You are designing the query routing layer for an internal Support Agent. To balance cost and accuracy, you want to avoid invoking an expensive LLM router for every query. However, you find that a purely Rule-Based (Regex) router is too brittle and fails on slight phrasing variations. What is the most robust architectural solution?

A. Implement a Hybrid Routing waterfall: Rule-Based -> Embedding-Based -> LLM-Based.

B. Expand the Rule-Based router's dictionary to include all possible synonyms generated by an LLM offline.

C. Use Step-Back Prompting to generalize all queries before applying Regex rules.

D. Route all queries through an intent classifier trained with a lightweight ML model.

Right answer: A
A sequential waterfall strategy systematically attempts the cheapest, fastest deterministic methods first, escalating to the semantic understanding of embeddings, and only paying the high inference cost when ambiguity necessitates it.

Before We Search: Securing the Perimeter

The Current State: Our Query Router just successfully mapped a complex user prompt to the exact correct database (e.g., finding the `HR_Docs_DB` or the `Live_Server_Logs`).

The Enterprise Dilemma: Three Massive Production Risks

SECURITY

"What if a Junior Engineer query is routed to HR, but they ask for the CTO's salary?"

EFFICIENCY

"What if 50 engineers ask the same question in 10 seconds? Do we pay for 50 searches?"

COST

"What if a recursive bug triggers 10,000 expensive LLM evaluations?"

The Solution: The Retrieval Gateway

A protective "bouncer" layer between routing logic and private company data.

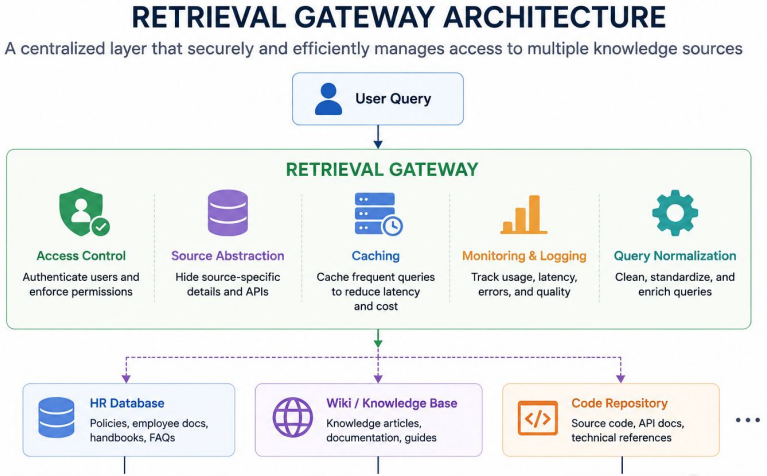
Retrieval Gateways: The Enterprise Bouncer

What it is:

A centralized security and management layer that sits between the Query Router and your actual knowledge sources (Vector DBs, APIs, SQL databases).

The "So What?" (Production Impact):

In a corporate environment, not all data is public, and not all AI queries are cheap. Without a gateway, sensitive executive documents could be exposed or recursive loops could drain cloud budgets.



Core Responsibilities:

Access Control (AuthN/AuthZ), Quota Management (FinOps), Semantic Caching, and Observability.

Access Control: Enforcing Least Privilege

What it is & How it Works

Concept: Ensuring the AI agent only retrieves documents the user has permissions to read.

Process: Gateway intercepts identity tokens (SSO/AD) and applies metadata filters in the Vector DB based on RBAC.

Scenario: Junior Engineer

"What are the latest salary bands?"

Gateway: BLOCKED

Reads user role, applies RBAC filter. Retrieval returns zero docs. LLM declines politely.

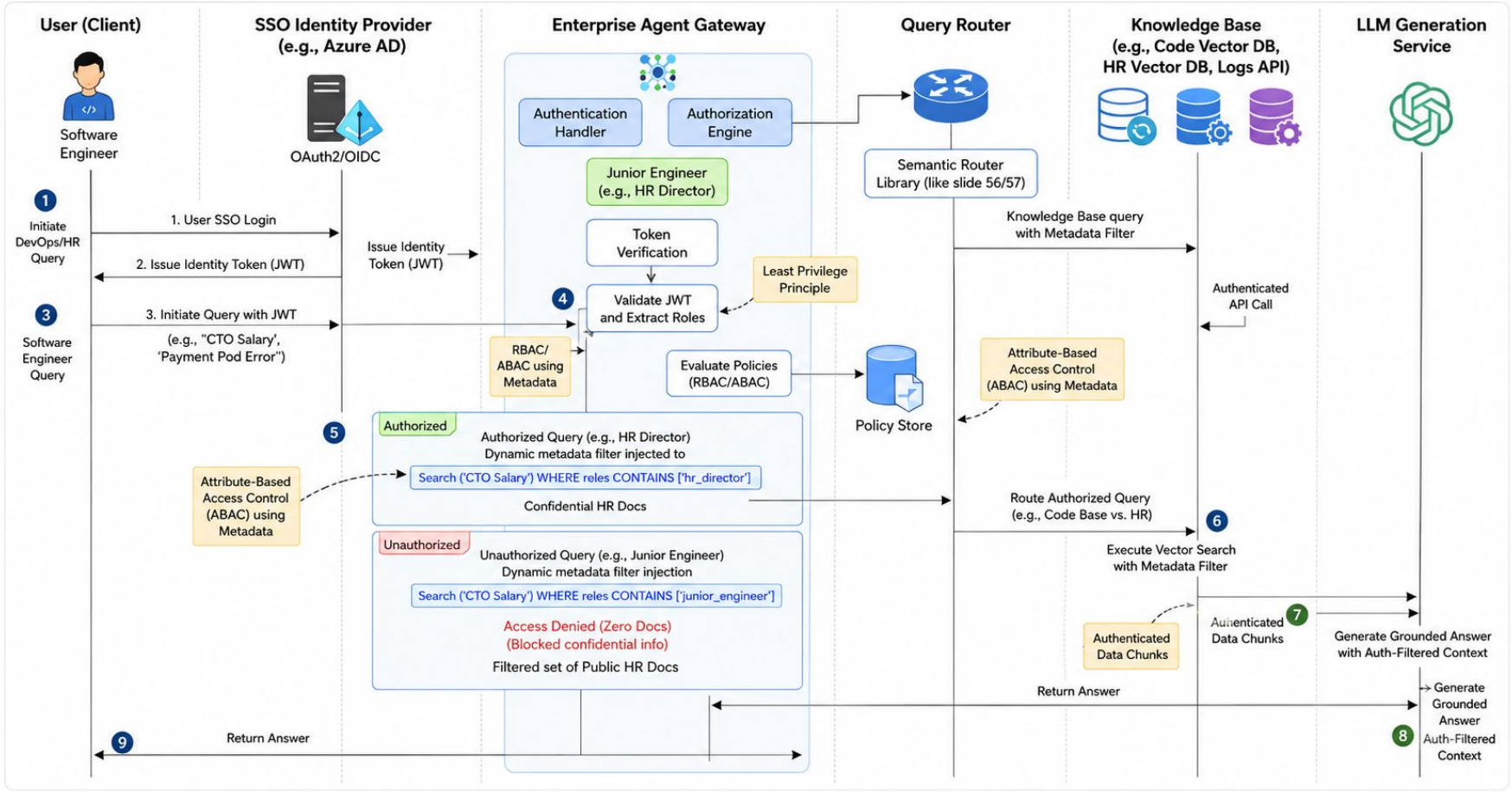
Scenario: HR Director

"What are the latest salary bands?"

Gateway: APPROVED

Authorizes query, retrieves sensitive PDF chunks, LLM summarizes findings.

Access Control: Enforcing Least Privilege




menti.com
2182 6296

You are building an Enterprise Support Agent with access to both public company wikis and highly confidential HR salary documents. To enforce Least Privilege and ensure a junior engineer cannot access executive salary data, where and how must the access control (AuthZ) be implemented?

A. In the Retrieval Gateway, by intercepting the user's identity token and injecting a dynamic metadata filter into the Vector DB query before retrieval.

B. By post-processing the retrieved chunks with a cross-encoder to filter out documents that do not match the user's security clearance level.

C. By using an LLM Router to classify the query intent as 'Sensitive,' and subsequently blocking the generation phase.

D. By instructing the Generation LLM via the system prompt to refuse answering if the context contains sensitive salary data.

Right answer: A
Injecting metadata filters at the database level guarantees that the retrieval engine physically cannot return unauthorized documents to the generation pipeline, creating a secure barrier against data leakage.

Cost Management: Protecting the Cloud Budget

What it is:

Monitoring token usage, API calls, and compute cycles to prevent runaway costs, especially in multi-agent frameworks.

How it is done:

The gateway tracks usage (user/team/project), enforces rate limits, throttles requests, and falls back to cheaper models.

Project Example: DevOps Agent

The Scenario

Agent runs intensive "LLM-as-a-Judge" loops on high-tier Vertex AI endpoints. Risk: accidental $30,000 credit burn due to recursive bugs.

Gateway Action

Enforces monthly token caps. At 90% budget, it throttles requests or routes non-critical tasks to cheaper local models.

Semantic Caching: Speed & Efficiency

What it is:

Storing answers to previous questions. If a new query means the *exact same thing semantically*, the gateway serves the cached answer instantly.

How it is done:

The gateway generates an embedding for the query and checks it against a fast, in-memory cache (like Redis).

Project Example: DevOps Agent

The Scenario

200 engineers ask variation of "Is US-East down?" within 5 minutes.

Gateway Action

Processes first query normally. Recognizes identical intent for next 199 and serves cached report.

The Impact

Latency drops from seconds to ms. Bypasses 199 expensive Vector DB lookups and LLM calls.

Semantic caching: choosing the threshold

A cache hit is a **similarity bet**. The threshold trades stale-answer risk against money saved.

The decision rule

Embed the query, find the nearest cached query, and reuse its answer only if similarity clears the threshold.

```
hit if cos(q, q_cache) >= tau
```

The asymmetry

The two errors are not equal. A miss costs one extra lookup; a false hit returns a wrong answer. Bias tau high.

```
cost(false hit) >> cost(miss)
```

What you save

If a fraction h of queries hit, you skip their full retrieval + LLM cost and drop latency to milliseconds.

```
savings = h . (retrieval+LLM)
```

In one line: set tau high enough that a hit really means the same question — the saved cost is large, but a confident wrong answer is expensive.

Observability: Tracking the Truth

What it is:

The gateway acts as a single pane of glass for all RAG activity. It decouples the Orchestration layer from the underlying databases and logs every action.

How it is done:

Every query, retrieved chunk, citation, latency metric, and cache hit is logged to a central dashboard (e.g., Datadog, Splunk).

Project Example: DevOps Agent

The Scenario

An engineer complains that the agent gave them outdated steps to restart a Kubernetes cluster.

Gateway Action

The platform team looks up the exact query ID and traces the outdated runbook returned by the Vector DB, allowing them to delete the document.

The Problem with "Single-Shot" RAG

Standard RAG operates as a strict, linear pipeline:

Embed Query → Retrieve Top-K → Generate Answer

Assumption 1: Perfect Retrieval

All necessary context can be found in a single database search.

Assumption 2: Complete Context

The retrieved chunks contain the *final* answer, not a clue pointing to another system.

The Reality of Production

Enterprise problems are rarely solved in one search. When a developer investigates an issue, they run a query, read the result, realize they need more context, and run a ***different*** query.

Linear RAG cannot reflect, pause, or loop back.

Case Study: The Multi-Hop Query

The Complex Task

"Find the error in the payment container, check if it's related to yesterday's auth module commit, and if so, draft a Slack update for the QA team."

How Linear RAG Fails

Vector Dilution

It tries to embed this entire prompt at once. The vector database gets confused by keywords from logs, codebases, and HR tools, returning irrelevant, blended results.

Missing Sequential Logic

The system needs to know the exact container error *before* it knows what to search for in the auth module commits. It cannot search both simultaneously.

Why We Can't Just "If/Else" Our Way Out

To solve these multi-hop queries, engineers often try to build massive, rule-based routing trees.

The Hardcoded Approach

IF query contains "error"
→ Search Logs

THEN IF Logs contain "auth"
→ Search Codebase

THEN IF Codebase contains "commit"
→ Draft Message

The Catch: Fragility

A highly brittle system.

Lacks semantic understanding and fails if exact keywords are missing. A developer typing *"payment pod crashed"* instead of *"error"* collapses the workflow.

We cannot hardcode a path for every possible debugging scenario.

The "So What?": From Passive Search to Active Problem Solving

The Enterprise Reality

In real-world software environments, resolving an incident is an iterative loop. It requires:

- Executing an initial search.
- Evaluating the result.
- Realizing a piece of context is missing.
- Planning a secondary action to fetch that missing piece.

The Necessary Shift

We cannot hardcode our way out of this complexity. To make our DevOps agent truly useful, we must shift the responsibility of ***how*** to solve the problem away from static code and hand it over to the LLM itself.

We need a system that doesn't just read data, but autonomously decides how to use our tools.


menti.com
2182 6296

You are implementing Semantic Caching in a Retrieval Gateway to intercept duplicate queries and reduce LLM inference costs. You must configure the cosine similarity threshold (tau) that triggers a cache hit. Given the asymmetry of errors in production—where a false hit (returning an incorrect cached answer) severely damages user trust, but a cache miss merely costs standard API latency/fees—what is the optimal design choice?

A. Implement an exact-match hash cache instead of semantic caching to completely eliminate the risk of false hits.

B. Bias the threshold high, accepting a lower overall cache hit rate and slightly higher baseline costs in exchange for guaranteeing accuracy on cached responses.

C. Bias the threshold extremely low to maximize the cache hit rate, saving the most money and compute resources possible across slight query variations.

D. Dynamically route all cache hits through a Cross-Encoder to verify accuracy before serving the cached response to the user.

Right answer: B
Because the cost of a false hit (wrong answer) is massively higher in terms of enterprise trust than the cost of a cache miss (paying for the LLM call), the threshold must be strict enough that a hit genuinely means the exact same question was asked.

Thank You

Instructor



Pawan Goyal

Teaching Assistants (IITKGP)



Manoj Balaji Jagadeeshan



Pretam Ray



Kunal Kingkar

Quiz

You are deploying a semantic search service for a dataset of 50 million document chunks. Your infrastructure has strict memory (RAM) constraints, but you can tolerate a slight drop in retrieval recall. Which indexing strategy is the most appropriate design choice?

A. IndexFlatIP

B. IVF + PQ (Inverted File + Product Quantization)

C. Exact Cross-Encoder Matching

D. HNSW (Hierarchical Navigable Small World)

Right answer: B
IVF reduces the search scope by clustering, and Product Quantization heavily compresses the vector memory footprint, directly addressing strict RAM limits for massive datasets.

An enterprise requires a retrieval pipeline that achieves accuracy close to a cross-encoder but strictly cannot afford the query-time latency of computing joint attention over all candidate document pairs. They are, however, willing to provision significantly more storage. Which architecture best satisfies these constraints?

A. A dual-stage Bi-encoder followed by Cross-encoder reranking

B. A standard Bi-encoder

C. BM25 Sparse Retrieval

D. ColBERT (Late Interaction)

Right answer: D
This model precomputes token-level embeddings, deferring interaction to a lightweight query-time MaxSim operation, which trades high storage costs for near cross-encoder accuracy without the heavy runtime compute.

A user repeatedly encounters 'no results' or hallucinated answers when asking highly specific, deep-in-the-weeds technical questions (e.g., 'Why can't I pull the switch association data for the rogue units on Floor 3?'). The vector database contains the underlying architectural rules that explain this limitation, but it lacks the highly specific keywords from the user's prompt. Which query transformation design pattern is specifically built to mitigate this failure mode?

A. Follow-up Question Transformation

B. Semantic Routing

C. Step-Back Prompting

D. Multi-Query Retrieval

Right answer: C
Step-Back Prompting uses an LLM to abstract a hyper-specific query into a broader, foundational question (e.g., 'What is the overall architecture of our network troubleshooting system?'), allowing the system to retrieve the underlying rules before answering the specific edge case.

You are designing the query routing layer for an internal Support Agent. To balance cost and accuracy, you want to avoid invoking an expensive LLM router for every query. However, you find that a purely Rule-Based (Regex) router is too brittle and fails on slight phrasing variations. What is the most robust architectural solution?

A. Implement a Hybrid Routing waterfall: Rule-Based -> Embedding-Based -> LLM-Based.

B. Expand the Rule-Based router's dictionary to include all possible synonyms generated by an LLM offline.

C. Use Step-Back Prompting to generalize all queries before applying Regex rules.

D. Route all queries through an intent classifier trained with a lightweight ML model.

Right answer: A
A sequential waterfall strategy systematically attempts the cheapest, fastest deterministic methods first, escalating to the semantic understanding of embeddings, and only paying the high inference cost when ambiguity necessitates it.

You are building an Enterprise Support Agent with access to both public company wikis and highly confidential HR salary documents. To enforce Least Privilege and ensure a junior engineer cannot access executive salary data, where and how must the access control (AuthZ) be implemented?

A. In the Retrieval Gateway, by intercepting the user's identity token and injecting a dynamic metadata filter into the Vector DB query before retrieval.

B. By post-processing the retrieved chunks with a cross-encoder to filter out documents that do not match the user's security clearance level.

C. By using an LLM Router to classify the query intent as 'Sensitive,' and subsequently blocking the generation phase.

D. By instructing the Generation LLM via the system prompt to refuse answering if the context contains sensitive salary data.

Right answer: A
Injecting metadata filters at the database level guarantees that the retrieval engine physically cannot return unauthorized documents to the generation pipeline, creating a secure barrier against data leakage.

You are implementing Semantic Caching in a Retrieval Gateway to intercept duplicate queries and reduce LLM inference costs. You must configure the cosine similarity threshold (tau) that triggers a cache hit. Given the asymmetry of errors in production—where a false hit (returning an incorrect cached answer) severely damages user trust, but a cache miss merely costs standard API latency/fees—what is the optimal design choice?

A. Implement an exact-match hash cache instead of semantic caching to completely eliminate the risk of false hits.

B. Bias the threshold high, accepting a lower overall cache hit rate and slightly higher baseline costs in exchange for guaranteeing accuracy on cached responses.

C. Bias the threshold extremely low to maximize the cache hit rate, saving the most money and compute resources possible across slight query variations.

D. Dynamically route all cache hits through a Cross-Encoder to verify accuracy before serving the cached response to the user.

Right answer: B
Because the cost of a false hit (wrong answer) is massively higher in terms of enterprise trust than the cost of a cache miss (paying for the LLM call), the threshold must be strict enough that a hit genuinely means the exact same question was asked.